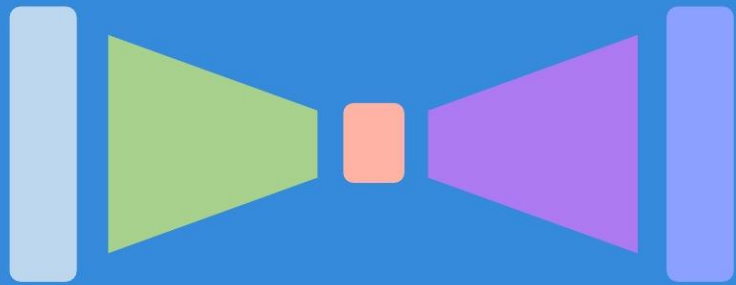
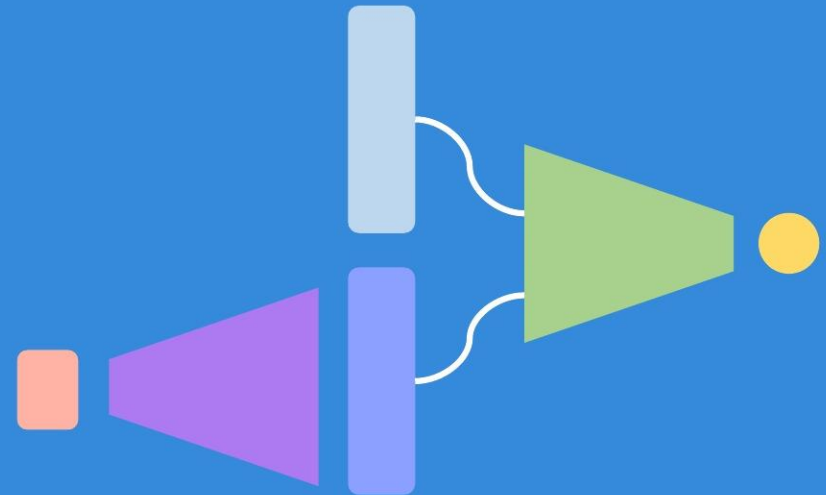


Latent variable models

Autoencoders and Variational
Autoencoders (VAEs)



Generative Adversarial
Networks (GANs)



What is a latent variable?



Myth of the Cave

What is a latent variable?

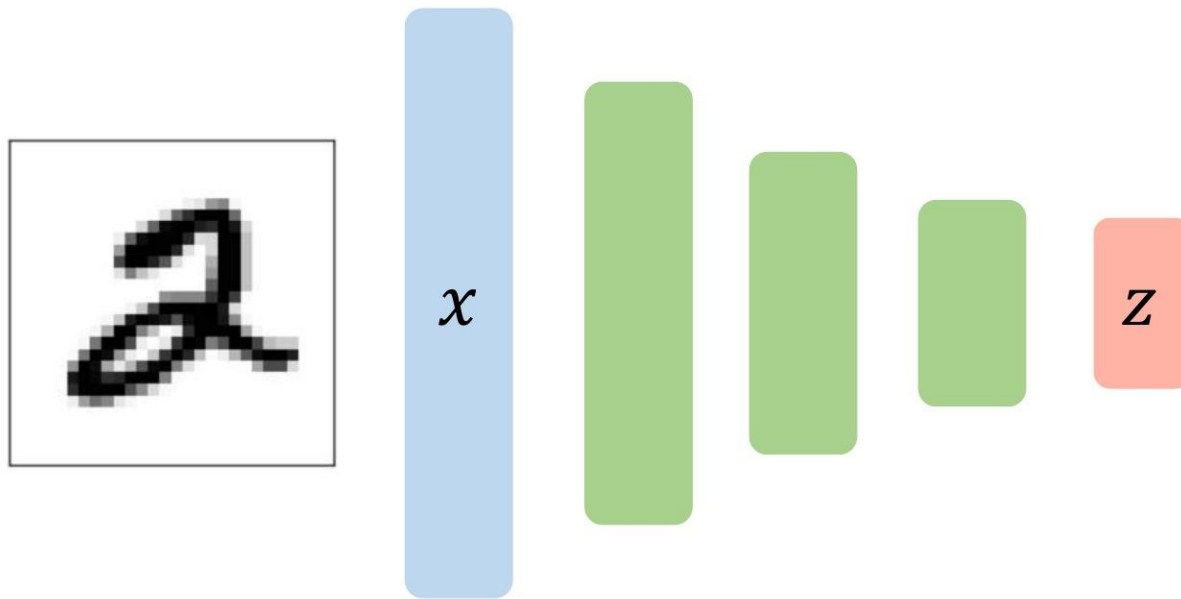


Can we learn the **true explanatory factors**, e.g. latent variables, from only observed data?

Autoencoders

Autoencoders: background

Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data



Why do we care about a low-dimensional z ?

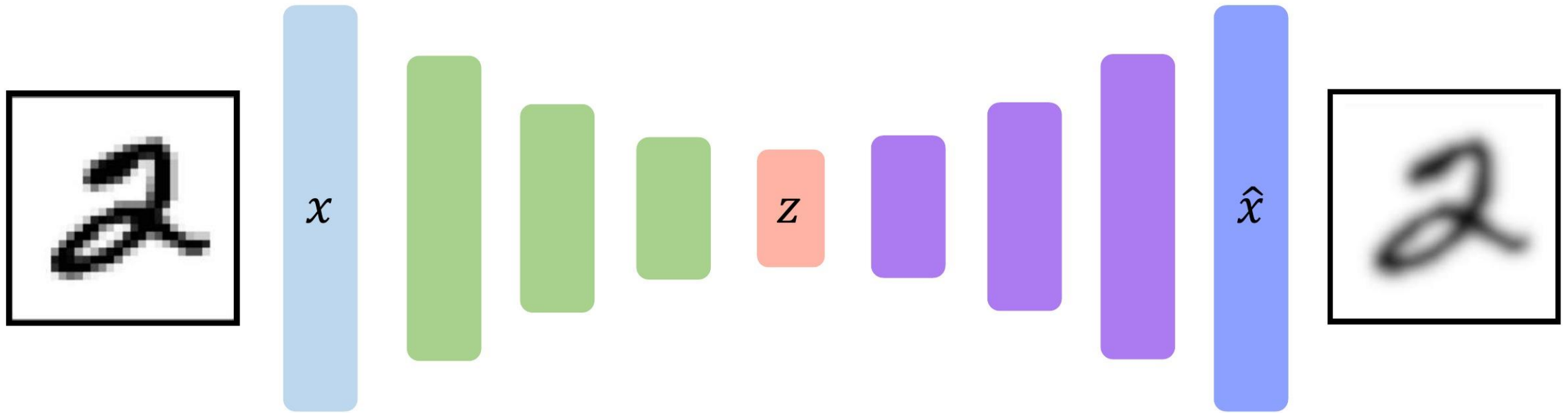


“Encoder” learns mapping from the data, x , to a low-dimensional latent space, z

Autoencoders: background

How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**

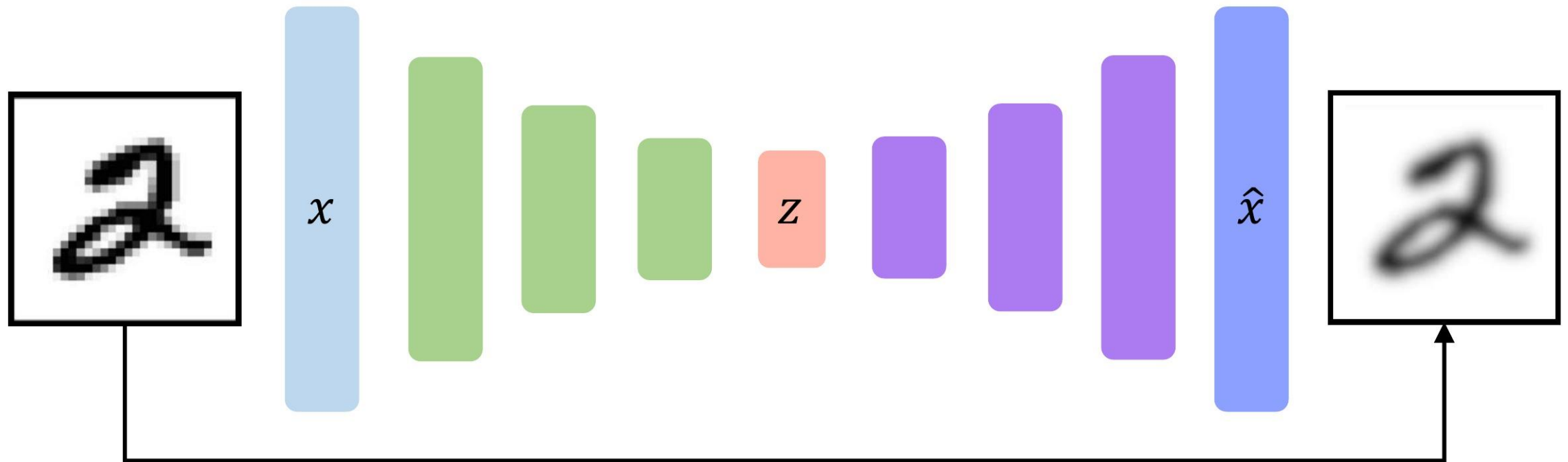


“Decoder” learns mapping back from latent, z , to a reconstructed observation, $\hat{x}$

Autoencoders: background

How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



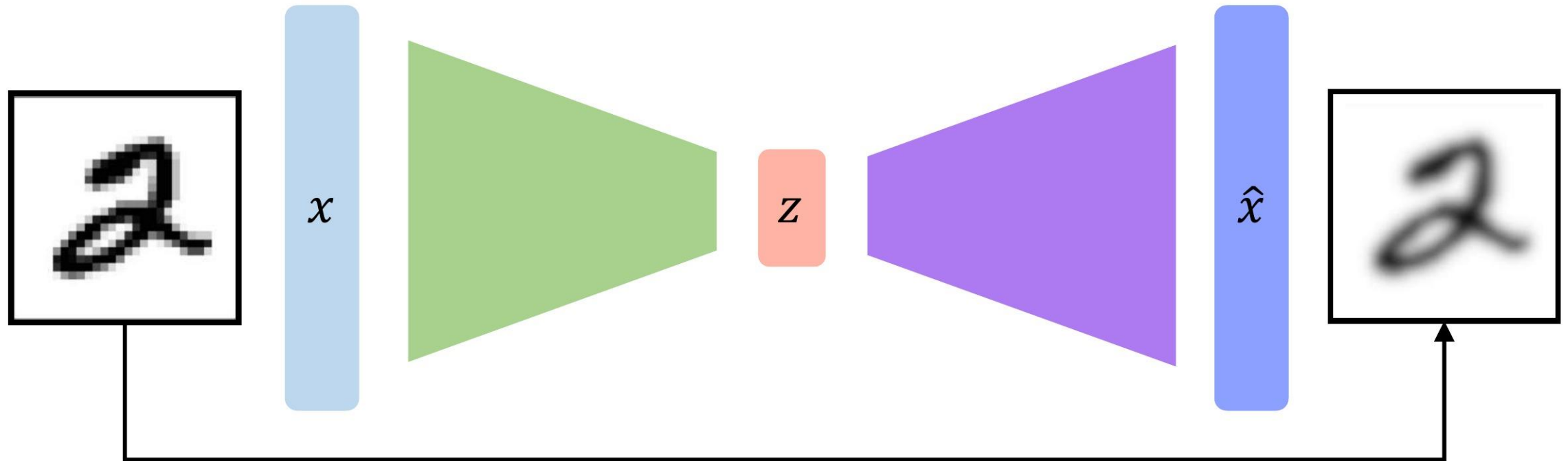
$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't
use any labels!!

Autoencoders: background

How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't
use any labels!!

Dimensionality of latent space → reconstruction quality

Autoencoding is a form of compression!

Smaller latent space will force a larger training bottleneck

2D latent space



5D latent space



Ground Truth



Autoencoders for representation learning

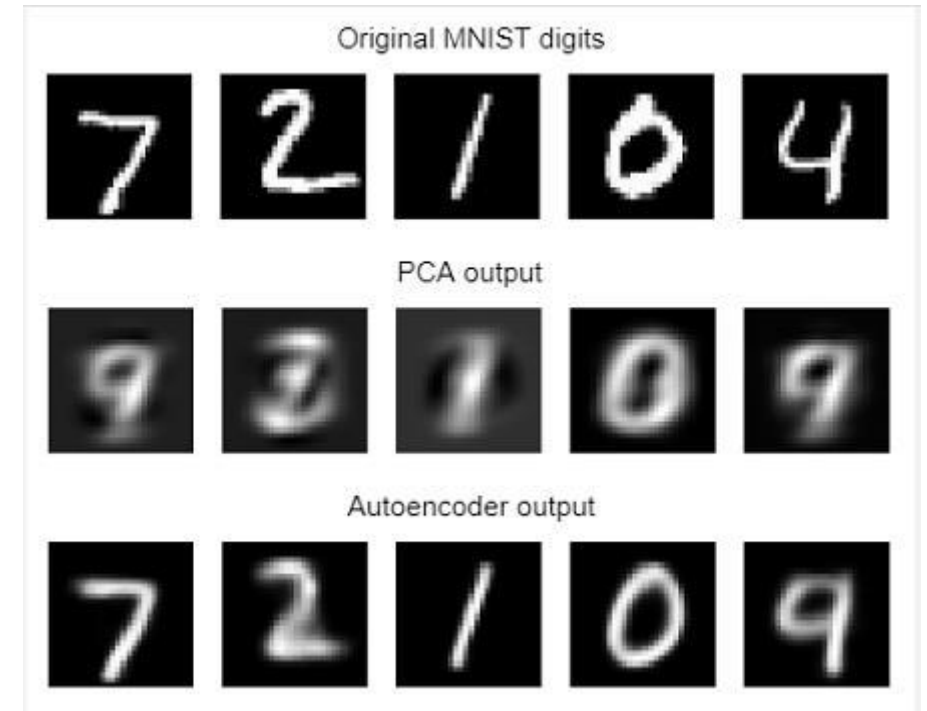
Bottleneck hidden layer forces network to learn a compressed latent representation

Reconstruction loss forces the latent representation to capture (or encode) as much “information” about the data as possible

Autoencoding = **Auto** automatically **encoding** data

Applications of AE

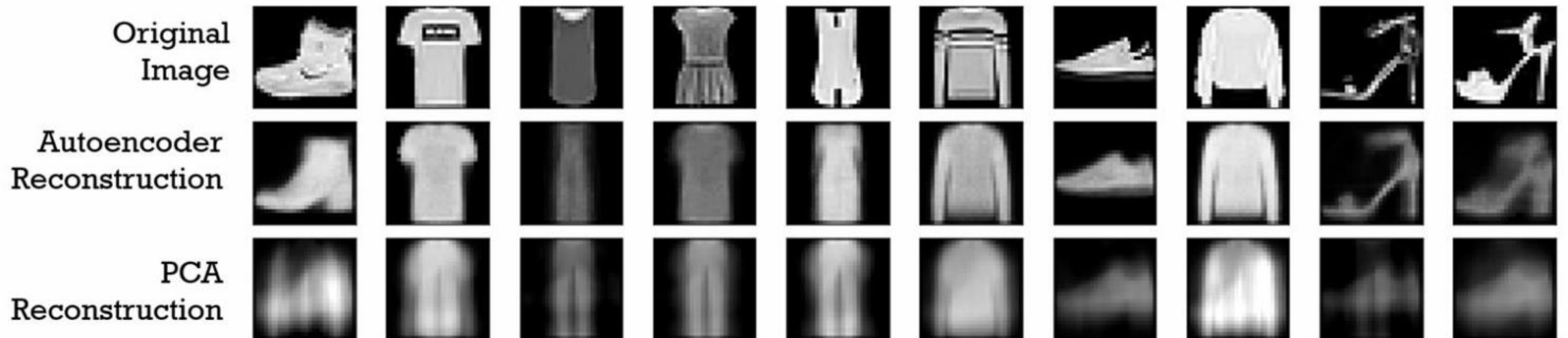
- Data Compression
- Image Retrieval
 - Makes search fast
 - Consumes less memory
- Image Denoising
- Image colorization
- Anomaly Detection



Applications of AE

Data Compression (AE vs PCA)

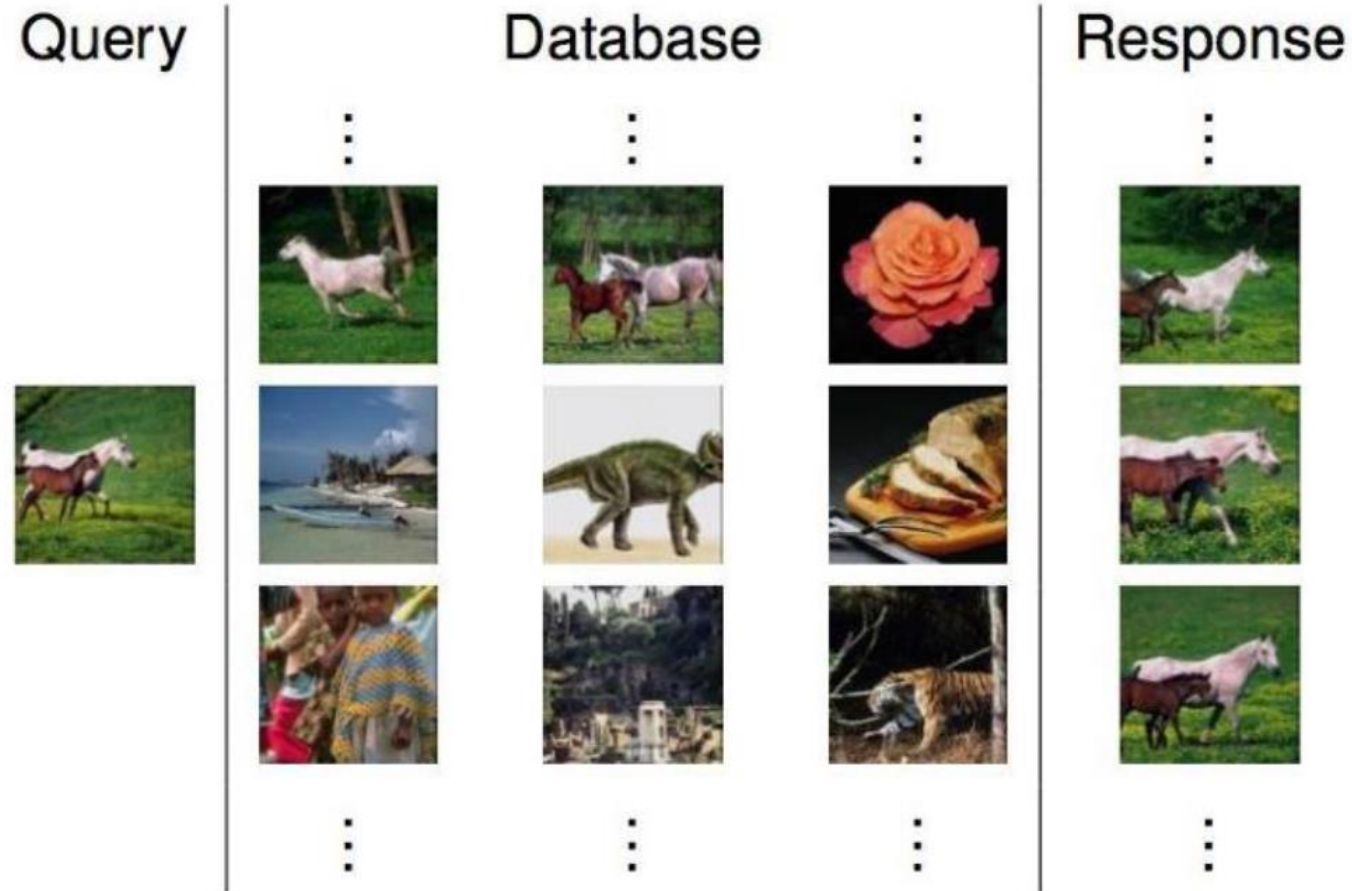
- Reconstruction
 - Latent vector of size 2
 - Compression from 28x28



Applications of AE

Image Retrieval

- Dimensionality reduction helps



Applications of AE

- Image Retrieval

- Once the autoencoder is trained, compute and **store the latent representations for all images** in the dataset.
- When a query image is provided, the encoder computes its latent representation z .
- This z is compared to the *stored representations* (using a similarity metric like Euclidean distance or cosine similarity).
- Makes search fast
- Consumes less memory

Applications of AE

- Image Denoising (DAE)

- Trained AE on image pairs (original (as target) and noisy (as input))
- Provide only noisy image during inference
- Decoder will compute difference with the original image and output an image similar to the original one (denoised)

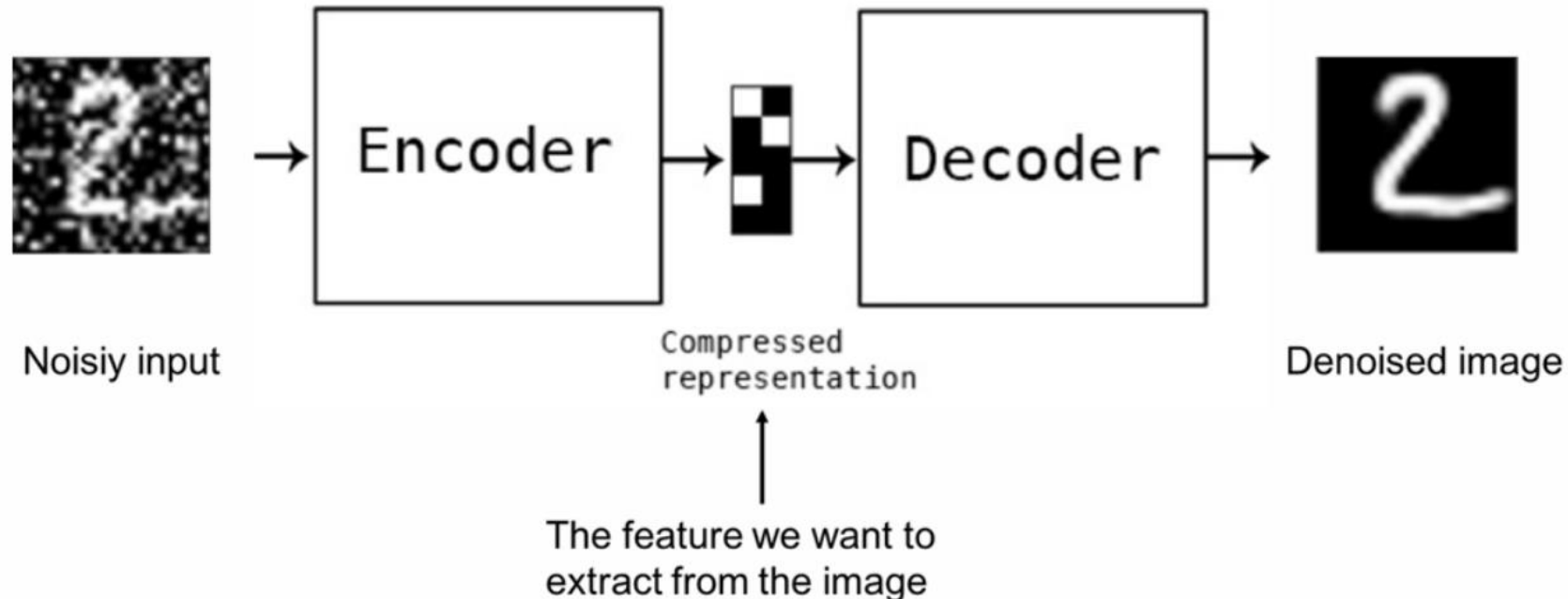
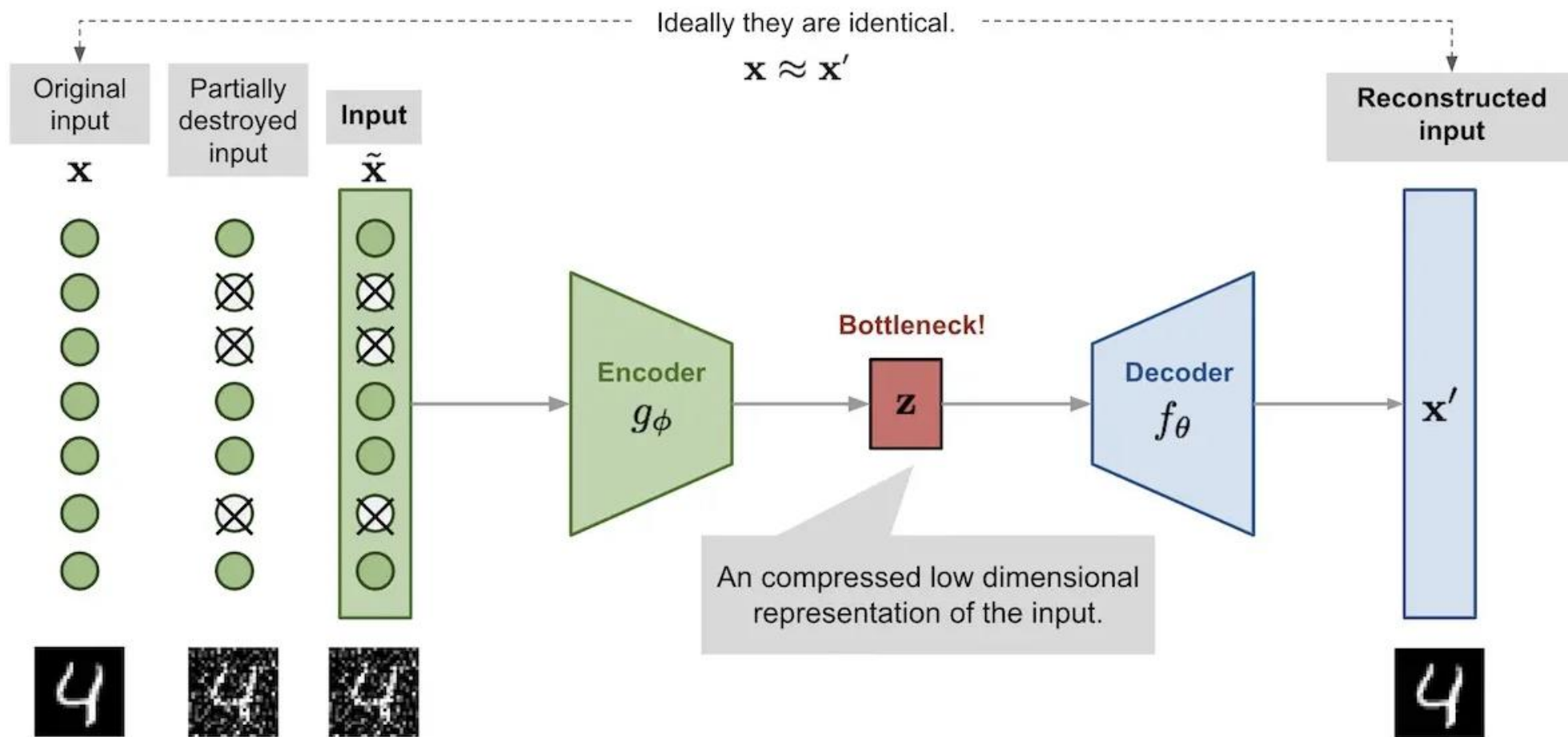


Image Denoising (DAE)



Denoising autoencoder with the corrupted input ([source](#))

Image Denoising (DAE)

Training phase:

- You **corrupt the input image with noise** (e.g., add Gaussian noise, salt-and-pepper).
- But the **target output is the original clean image**.
- So the AE learns to *map noisy* → *clean*.
- Encoder compresses noisy input into latent representation.
- Decoder reconstructs the clean version.

Inference phase:

- Give the trained model a new noisy image.
- The encoder “filters out” noise in latent representation (because it has learned the general data distribution of clean images).
- Decoder reconstructs the closest clean image → **denoised output**.

Key idea: during training, AE doesn't learn to just “reproduce input,” it learns to *ignore irrelevant noise* because target is always clean.

Applications of AE

- **Image colorization**
 - Input grayscale, output colored image
 - To train the model, paired grayscale and color images are required. The grayscale image serves as the input, while the corresponding color image serves as the target for reconstruction.



Anomaly Detection



When an anomalous image (that is different from a trained image) is input, the AE will not be able to reconstruct it well because it has not seen such patterns during training. Hence, the reconstruction error will be high.

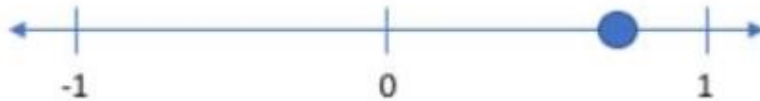
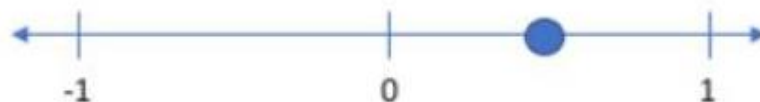
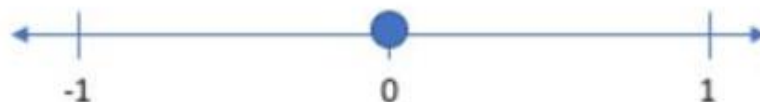
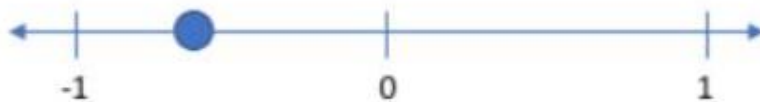
A threshold is set, and any input with a reconstruction error above this threshold is flagged as an anomaly.

Can we generate new data with Autoencoders?

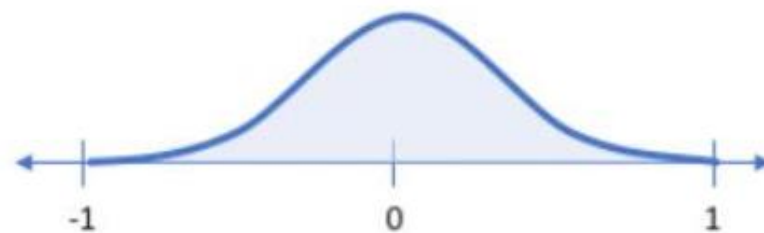
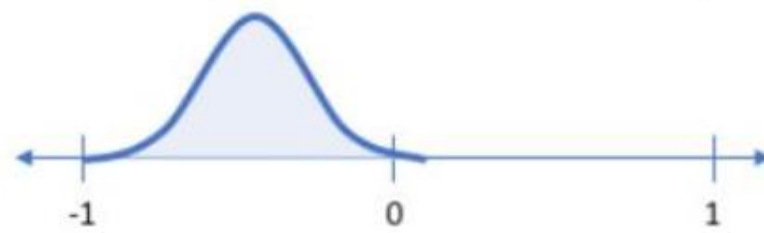
Variational Autoencoders (VAEs)



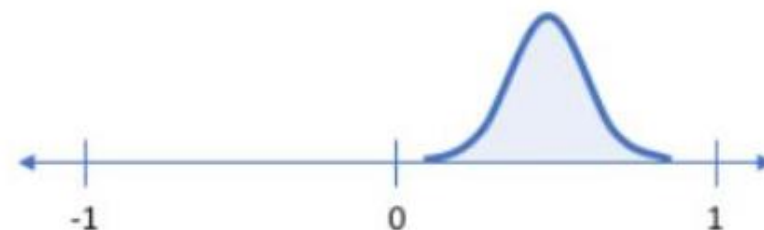
Smile (discrete value)

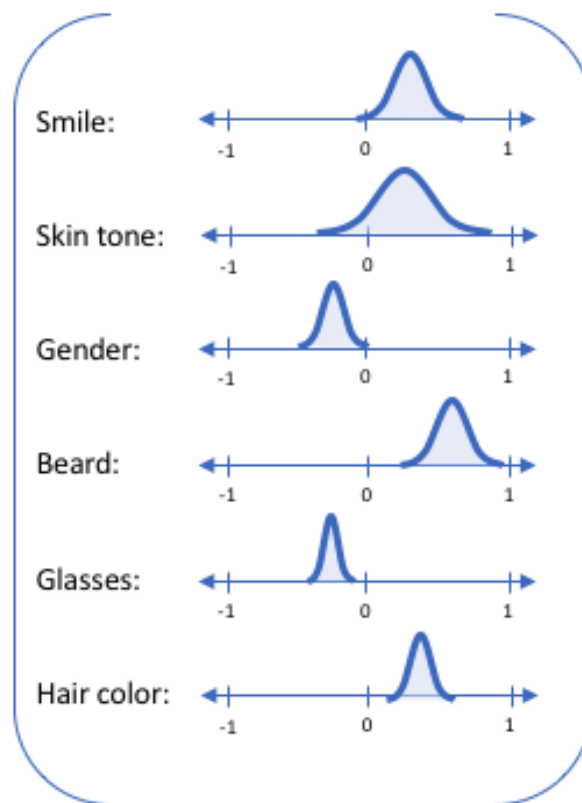
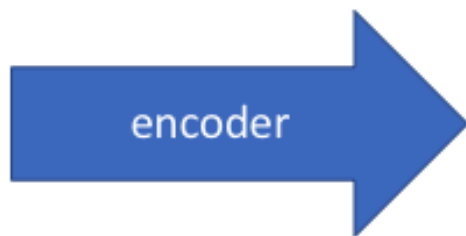


Smile (probability distribution)

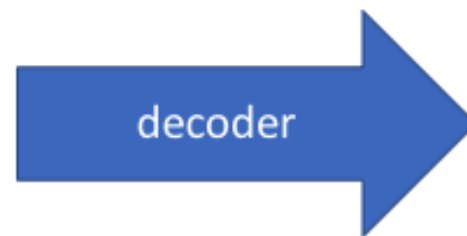


VS.

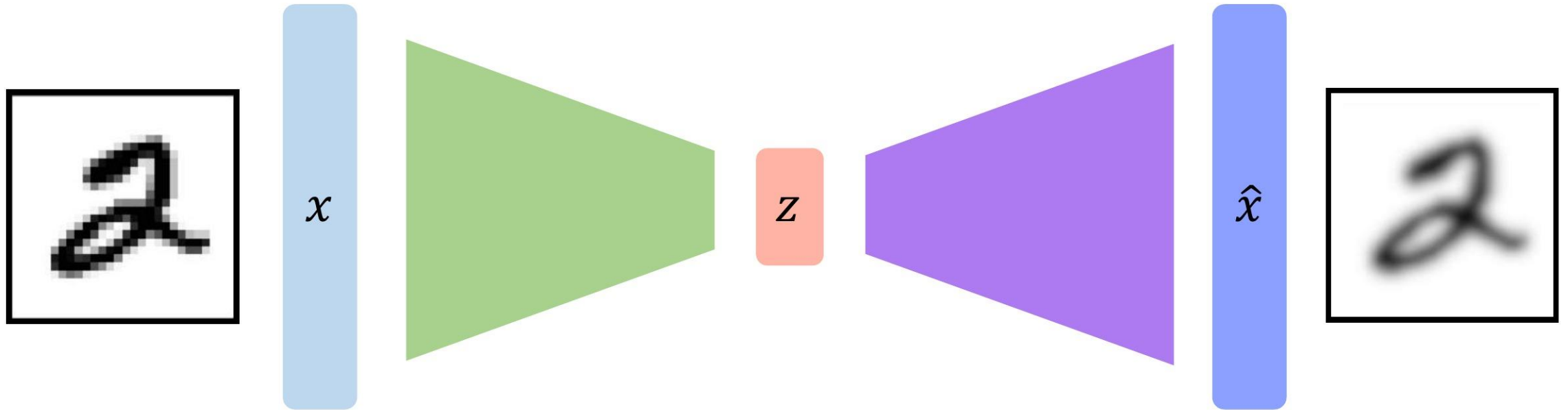




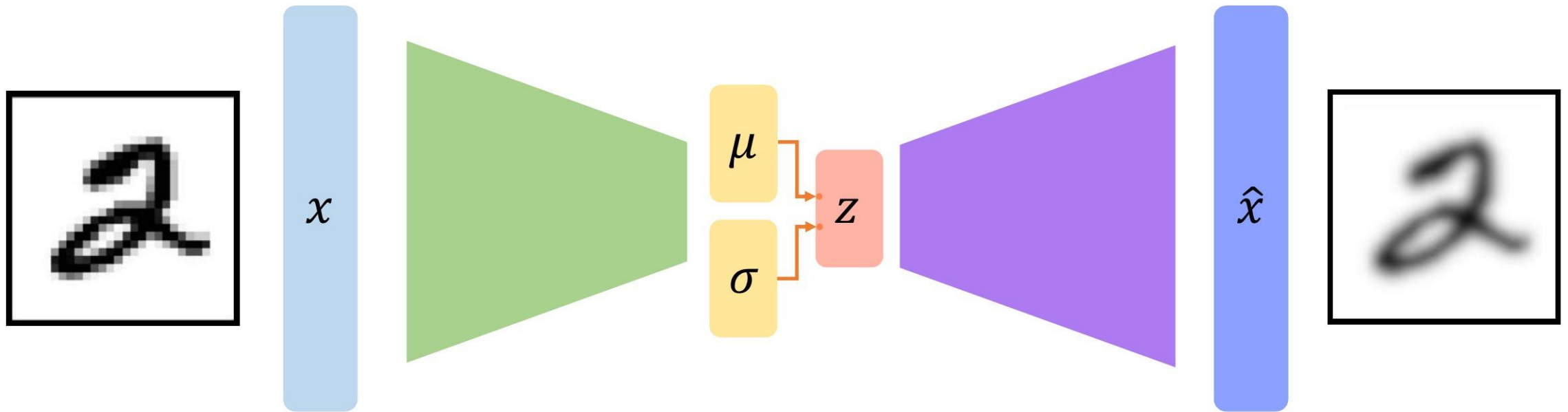
Latent attributes



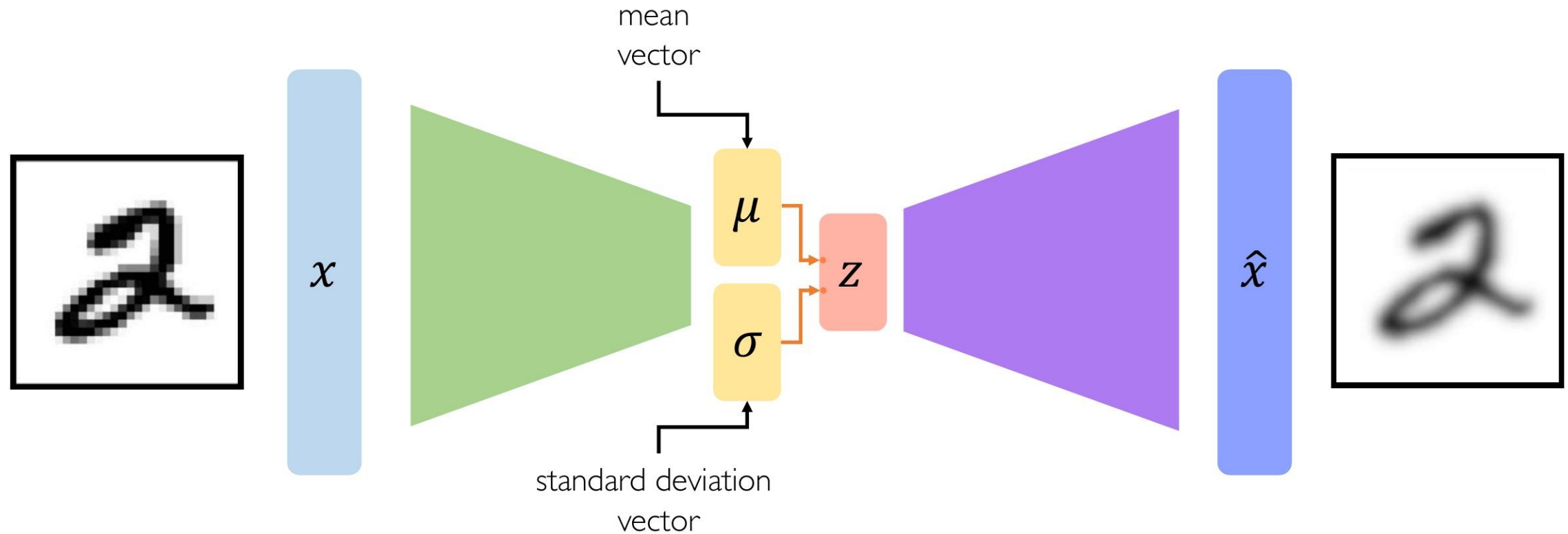
VAEs: key difference with traditional autoencoder



VAEs: key difference with traditional autoencoder

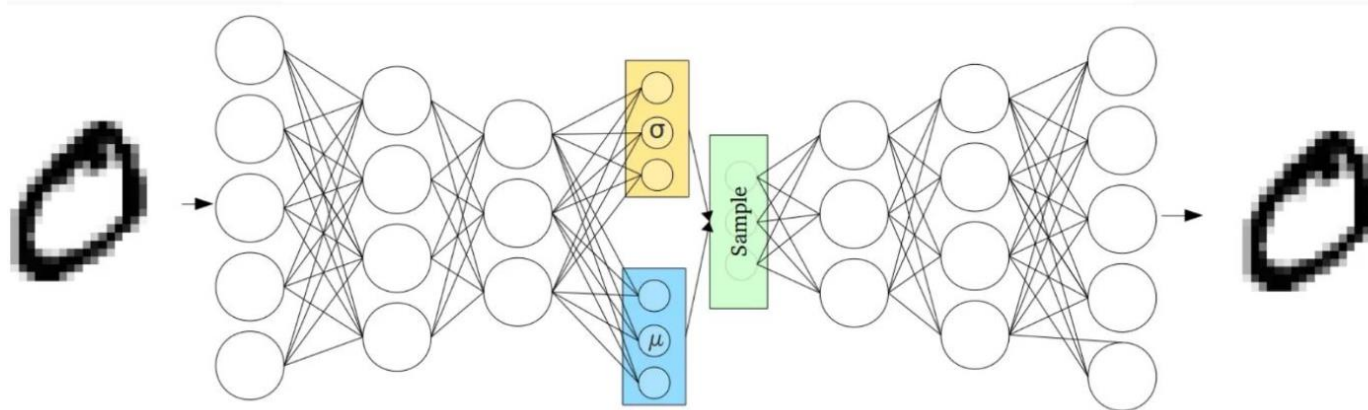


VAEs: key difference with traditional autoencoder

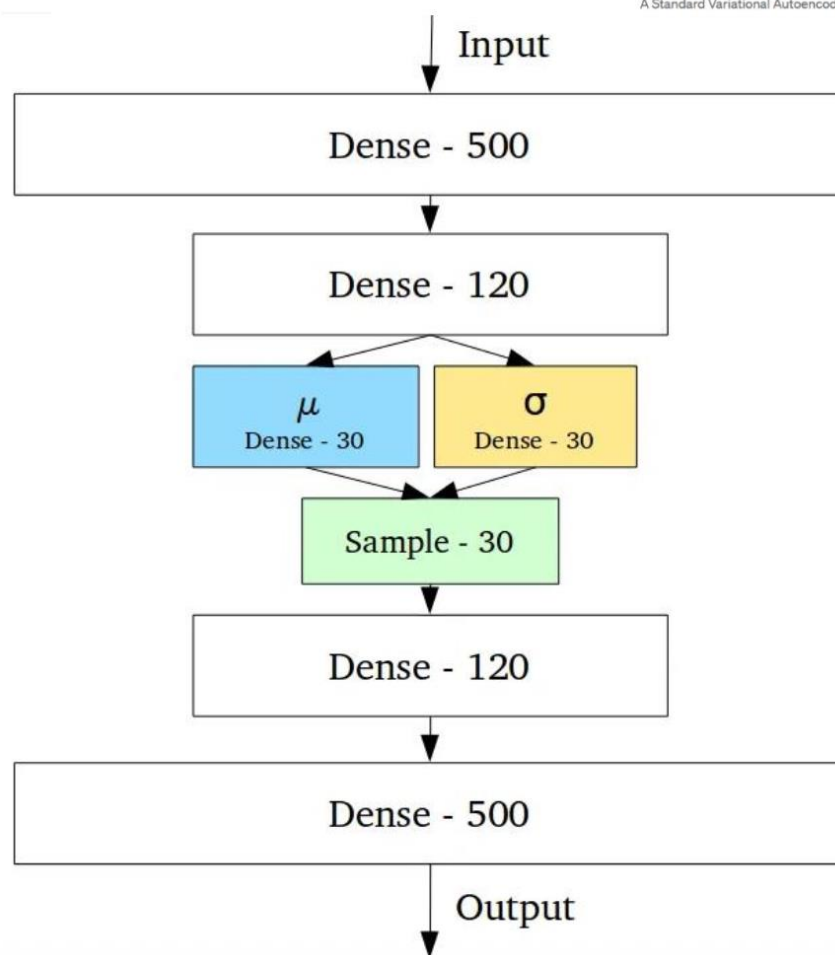


Variational autoencoders are a probabilistic twist on autoencoders!

Sample from the mean and standard dev. to compute latent sample



A Standard Variational Autoencoder



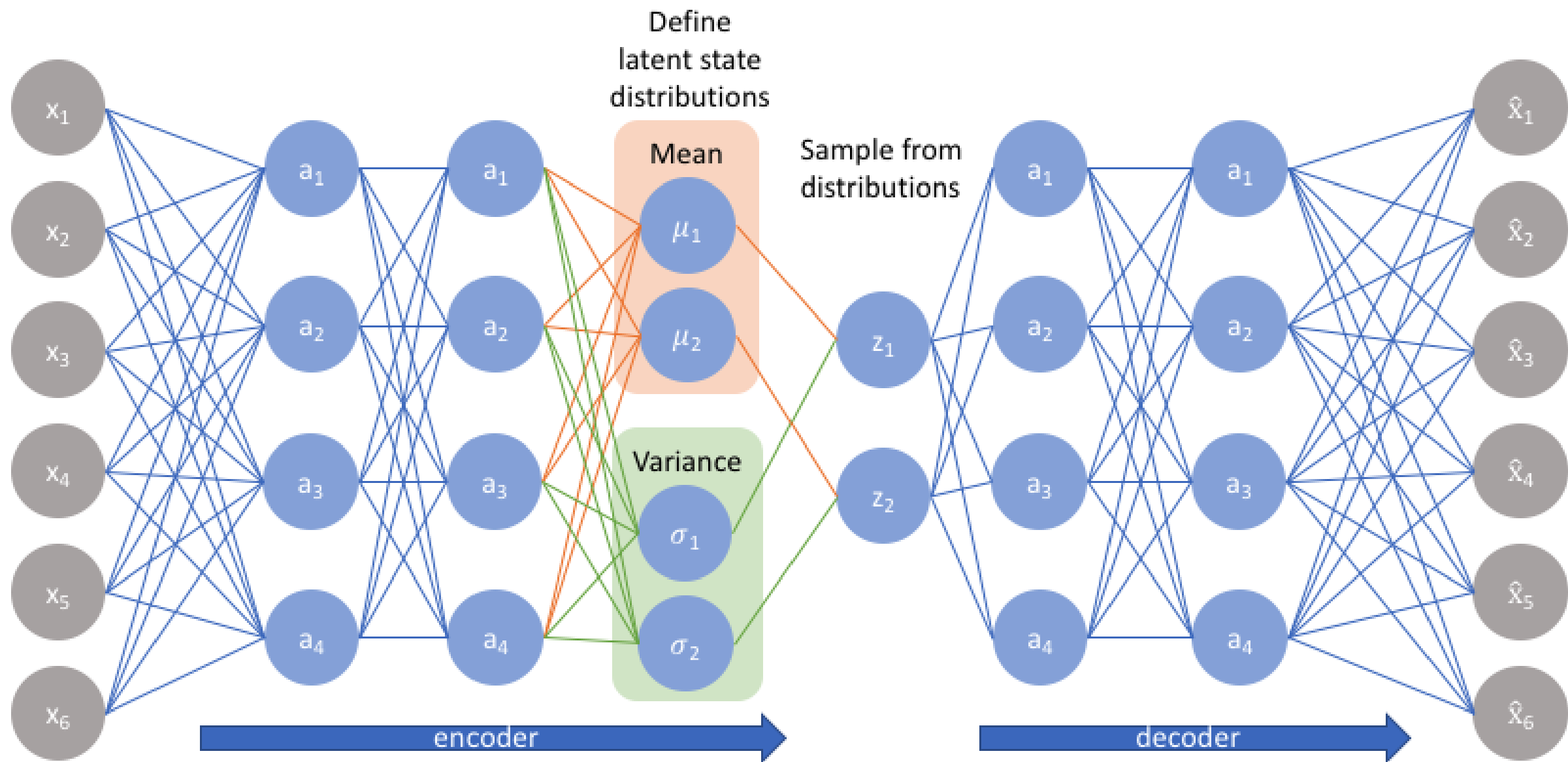
They form the parameters of a vector of random variables of length a , with

Output μ [0.1, 1.2, 0.2, 0.8,...]

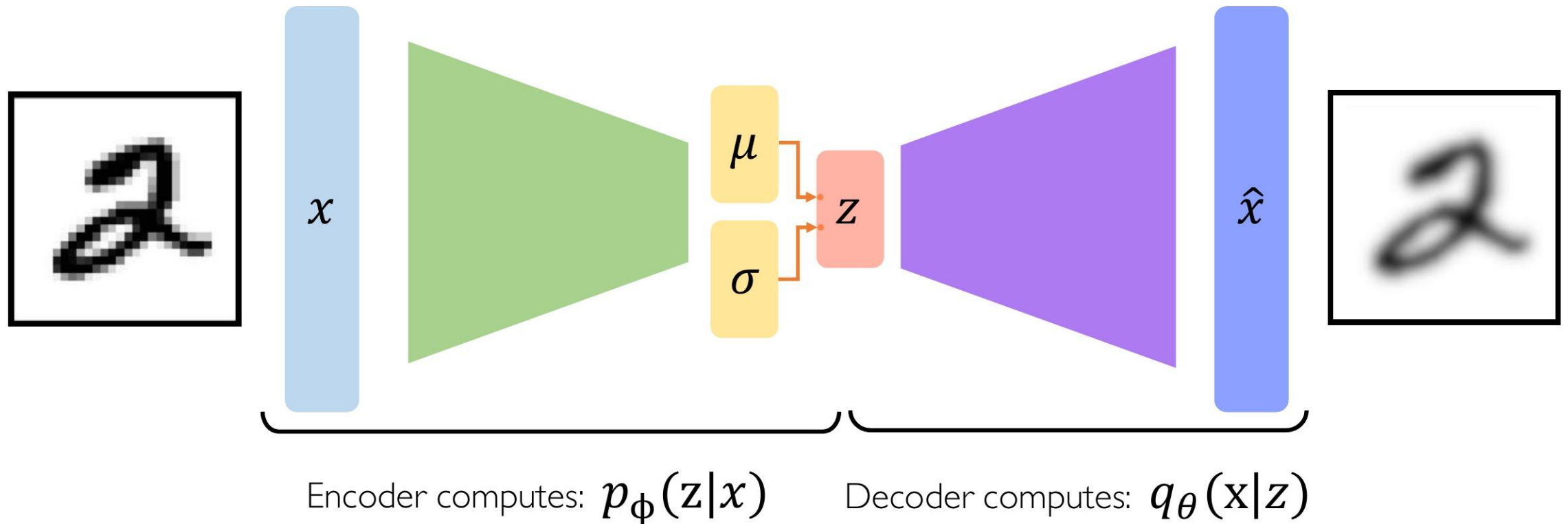
Output σ [0.2, 0.5, 0.8, 1.3,...]

Intermediate $\mathbf{X}$ [$X_1 \sim N(0.1, 0.2^2)$, $X_2 \sim N(1.2, 0.5^2)$, $X_3 \sim N(0.2, 0.8^2)$, $X_4 \sim N(0.8, 1.3^2)$, ...]

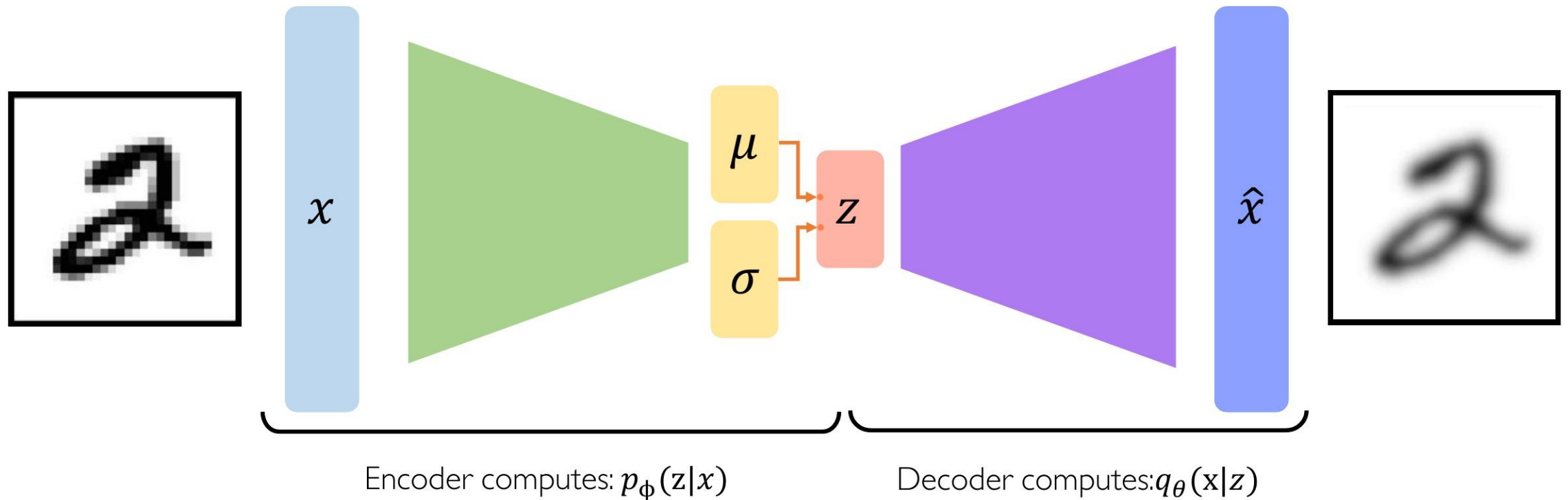
↓ sample
Sampled vector [0.28, 1.65, 0.92, 1.98,...]



VAE optimization

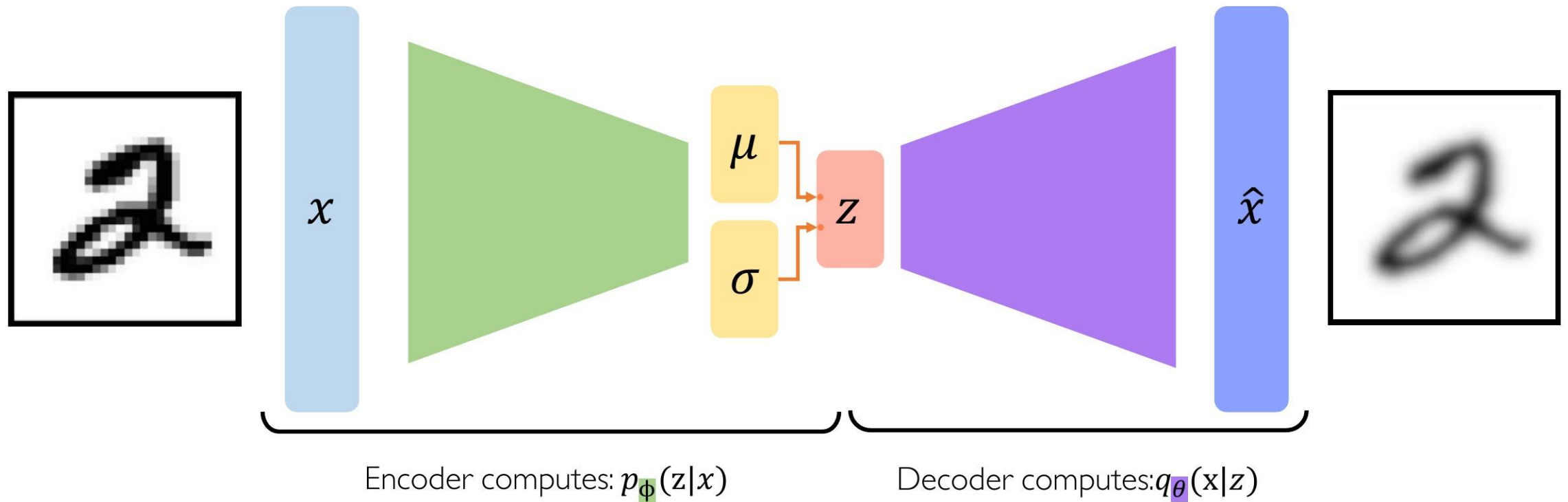


VAE optimization



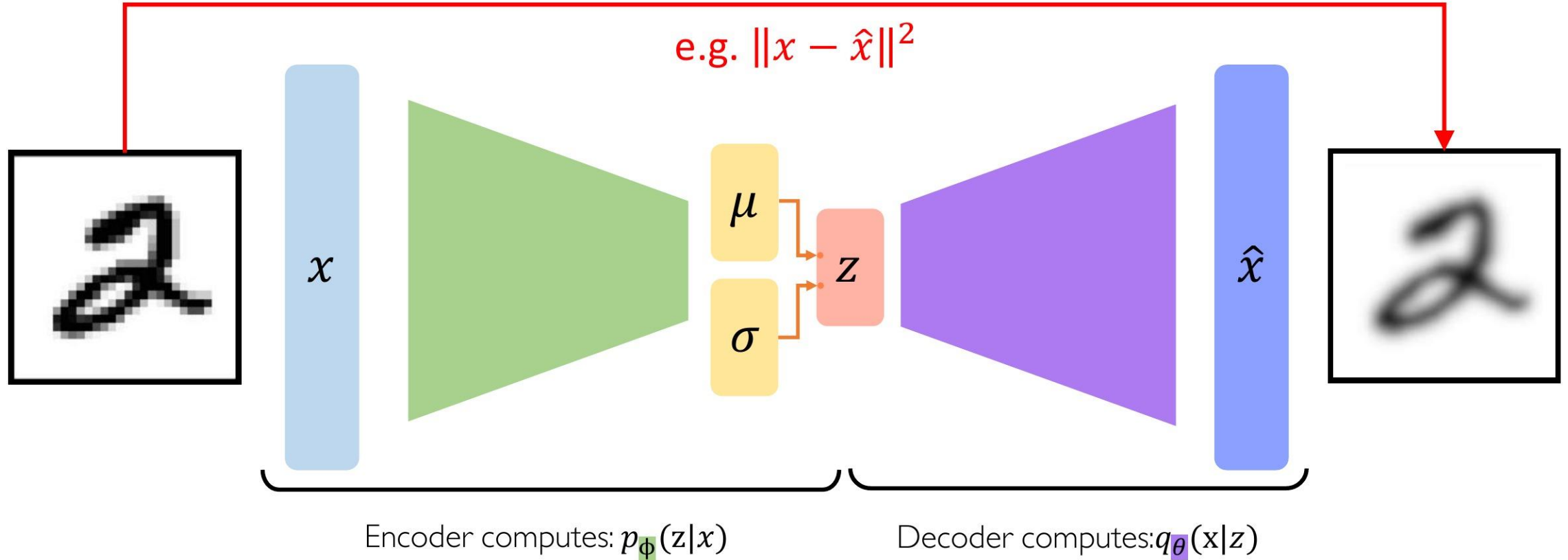
$$\mathcal{L}(\phi, \theta) = (\text{reconstruction loss}) + (\text{regularization term})$$

VAE optimization



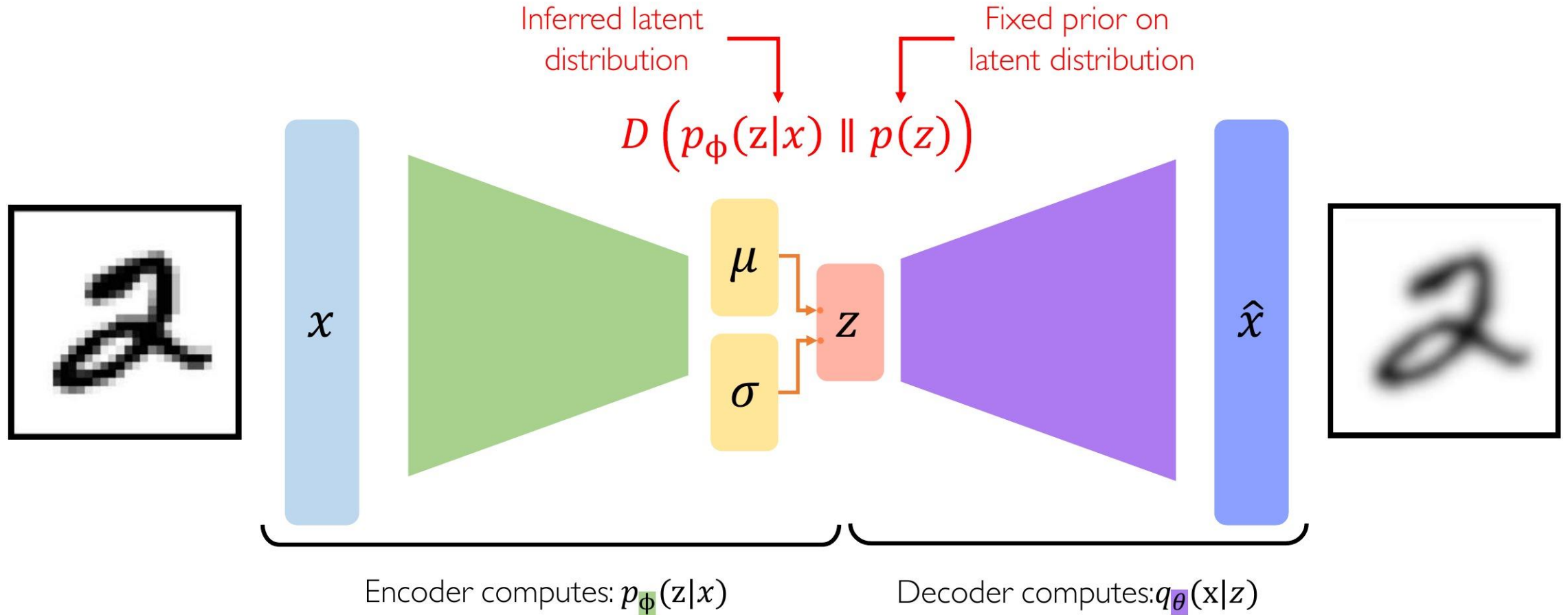
$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

VAE optimization



$$\mathcal{L}(\phi, \theta, x) = \text{(reconstruction loss)} + \text{(regularization term)}$$

VAE optimization

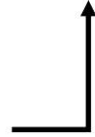


$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

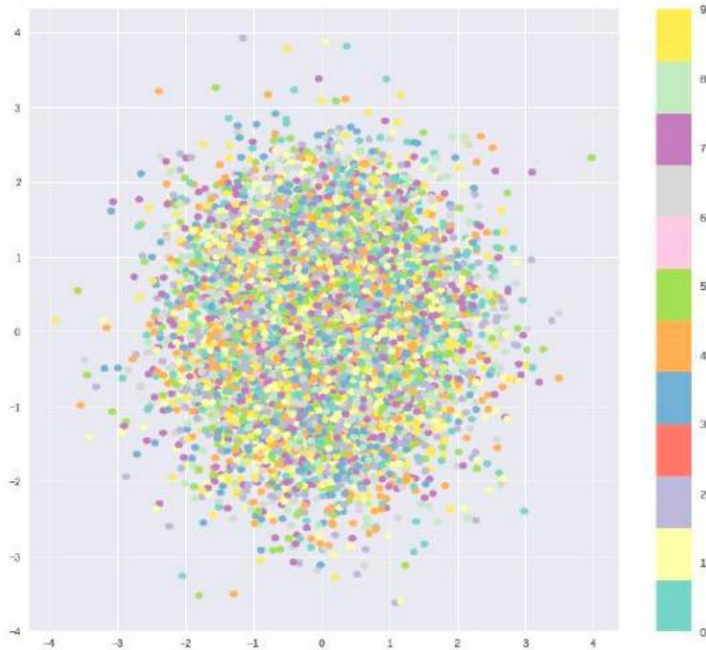
Priors on the latent distribution

$$D \left(p_{\phi}(z|x) \parallel p(z) \right)$$

Inferred latent
distribution



Fixed prior on
latent distribution



Common choice of prior:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

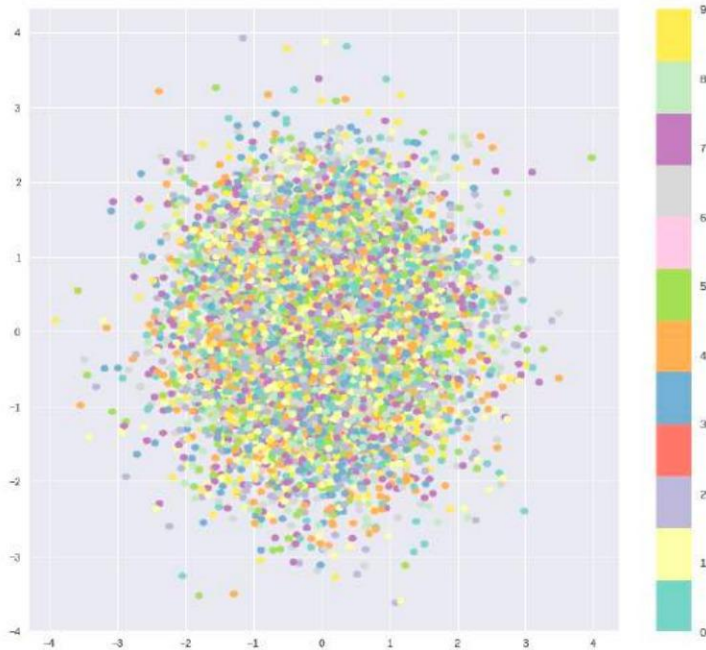
- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (ie. memorizing the data)

Priors on the latent distribution

$$D \left(p_{\phi}(z|x) \parallel p(z) \right)$$

$$= -\frac{1}{2} \sum_{j=0}^{k-1} (\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence between
the two distributions



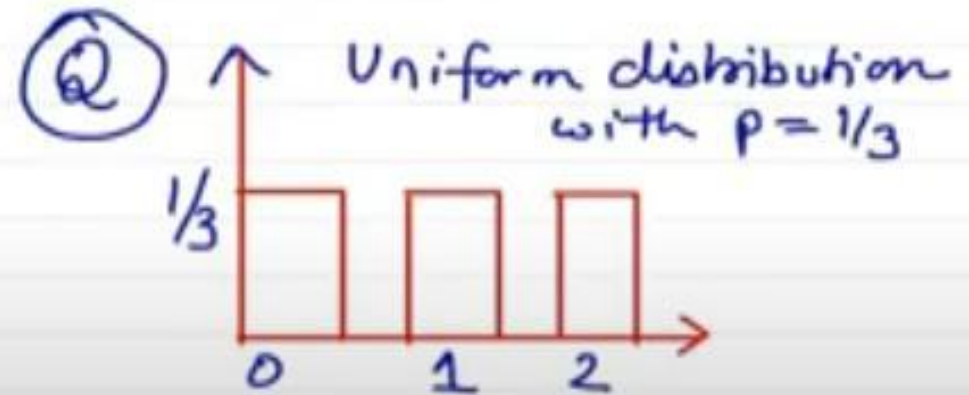
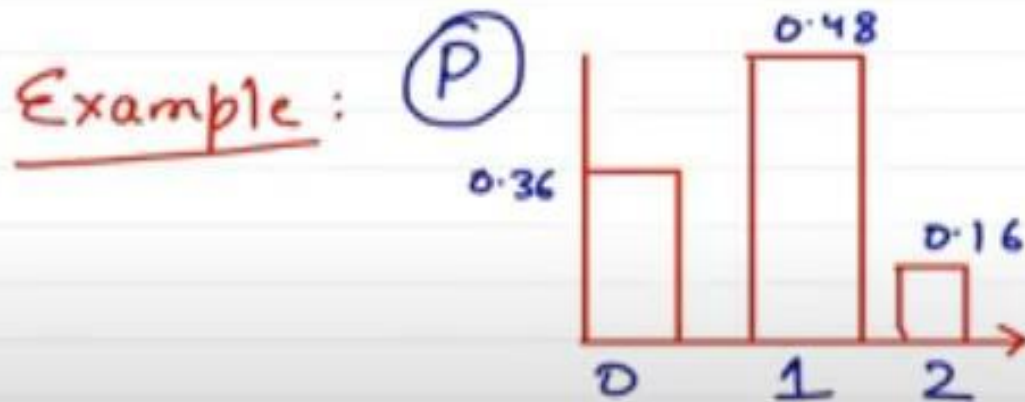
Common choice of prior:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (ie. memorizing the data)

KL divergence

$$D_{KL}(P \parallel Q) = \sum_x P(X=x) \log \left(\frac{P(X=x)}{Q(X=x)} \right)$$
$$= \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$



$$D_{KL}(Q \parallel P) = \frac{1}{3} \ln \left(\frac{0.333}{0.36} \right) + \frac{1}{3} \ln \left(\frac{0.333}{0.48} \right) + \frac{1}{3} \ln \left(\frac{0.333}{0.16} \right)$$

KL divergence

Properties:

① $KL(P \parallel Q) \text{ or } KL(Q \parallel P) \geq 0 \quad \checkmark$

② $KL(P \parallel Q) \neq KL(Q \parallel P) \quad (\text{Not symmetric})$

Deriving the KL term in the VAE loss

From the definition of KL divergence to the closed form you implement

$$D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z)) = \frac{1}{2} \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$$

the destination

The two distributions, and one useful fact

SETUP

ENCODER GIVES $q_{\phi}(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ with $\mu, \sigma^2 \in \mathbb{R}^k$

PRIOR $p(z) = \mathcal{N}(0, I)$ (the common choice: mean 0, variance 1 in every dimension)

Both are diagonal Gaussians, so they factorise across the k latent dimensions — and KL divergence of a product is the sum of the KLs:

$$D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z)) = \sum_{j=0 \dots k-1} D_{\text{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \parallel \mathcal{N}(0, 1))$$

So the whole problem reduces to one dimension. Derive it once, then sum over j .

Write down what KL divergence means

STEP 1 · DEFINITION

DEFINITION

$$D_{\text{KL}}(q \parallel p) = \int q(z) \log [q(z) / p(z)] dz = \mathbb{E}_{z \sim q} [\log q(z) - \log p(z)]$$

It is an expectation under q of the log-ratio. So we need the two log-densities. For a one-dimensional Gaussian:

ENCODER $q(z) = \mathcal{N}(\mu, \sigma^2)$

$$\log q(z) = -\frac{1}{2} \log(2\pi\sigma^2) - (z - \mu)^2 / (2\sigma^2)$$

PRIOR $p(z) = \mathcal{N}(0, 1)$

$$\log p(z) = -\frac{1}{2} \log(2\pi) - z^2 / 2$$

Both come straight from the Gaussian density, with the second one simplified using $\mu = 0$ and $\sigma^2 = 1$.

Subtract the two log-densities

STEP 2 · SUBTRACT

DIFFERENCE $\log q(z) - \log p(z) = -\frac{1}{2} \log(2\pi\sigma^2) - (z-\mu)^2/(2\sigma^2) + \frac{1}{2} \log(2\pi) + z^2/2$

THE 2π CANCELS $-\frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2} \log(2\pi) = -\frac{1}{2} \log \sigma^2$

$$\log q(z) - \log p(z) = -\frac{1}{2} \log \sigma^2 - (z - \mu)^2 / (2\sigma^2) + z^2 / 2$$

Three pieces, and each one is easy to average under q :

a constant, a squared deviation from the mean, and a raw second moment. That is the next step.

Average each piece under $z \sim \mathcal{N}(\mu, \sigma^2)$

STEP 3 · TAKE THE EXPECTATION

$$\mathbb{E}[-\tfrac{1}{2} \log \sigma^2] = -\tfrac{1}{2} \log \sigma^2$$

It does not depend on z — a constant comes straight out.

$$\mathbb{E}[(z - \mu)^2 / (2\sigma^2)] = \sigma^2 / (2\sigma^2) = \tfrac{1}{2}$$

Because $\mathbb{E}[(z - \mu)^2]$ is the variance, σ^2 , by definition.

$$\mathbb{E}[z^2 / 2] = (\sigma^2 + \mu^2) / 2$$

Because $\mathbb{E}[z^2] = \text{Var}(z) + (\mathbb{E}[z])^2 = \sigma^2 + \mu^2$.

The middle line is the one that does the work: the messy $(z - \mu)^2$ term collapses to the constant $\tfrac{1}{2}$, and every trace of z disappears. What is left contains only μ and σ^2 .

Put the three pieces back together

STEP 4 · COLLECT

ADD THEM UP

$$D_{\text{KL}} = -\frac{1}{2} \log \sigma^2 - \frac{1}{2} + (\sigma^2 + \mu^2) / 2$$

$$D_{\text{KL}} = \frac{1}{2} (\sigma^2 + \mu^2 - 1 - \log \sigma^2)$$

one dimension

Now sum over the k latent dimensions — which the factorisation on the setup slide allows:

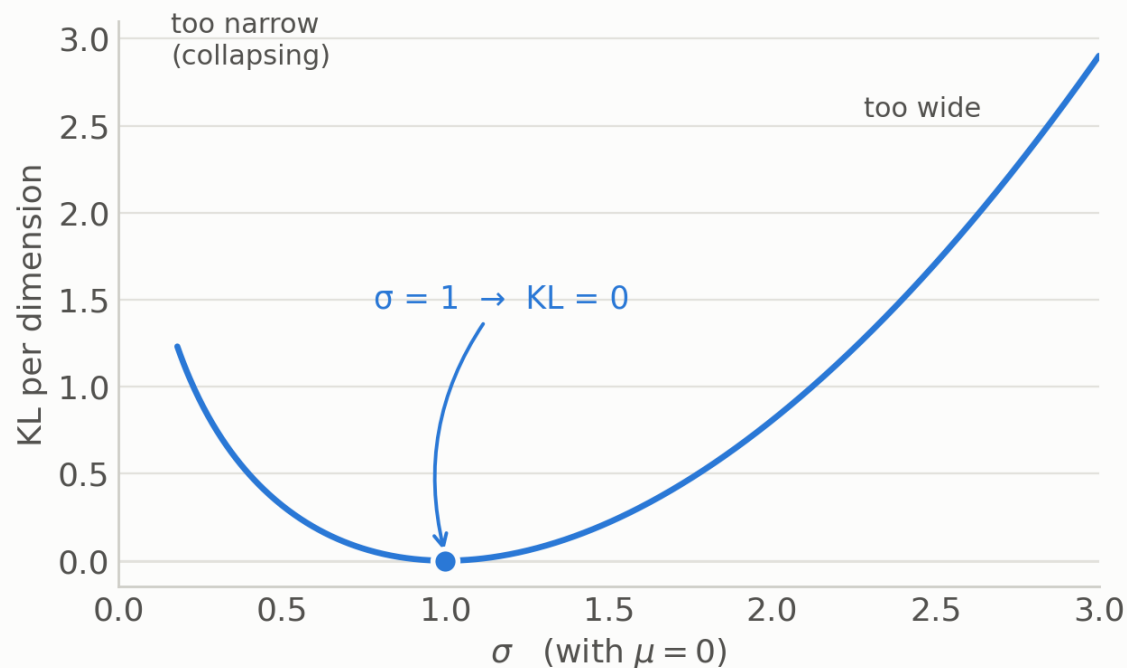
$$D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z)) = \frac{1}{2} \sum_{j=0 \dots k-1} (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$$

Equivalently, factoring the minus sign out: $D_{\text{KL}} = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$ — the form most code uses.

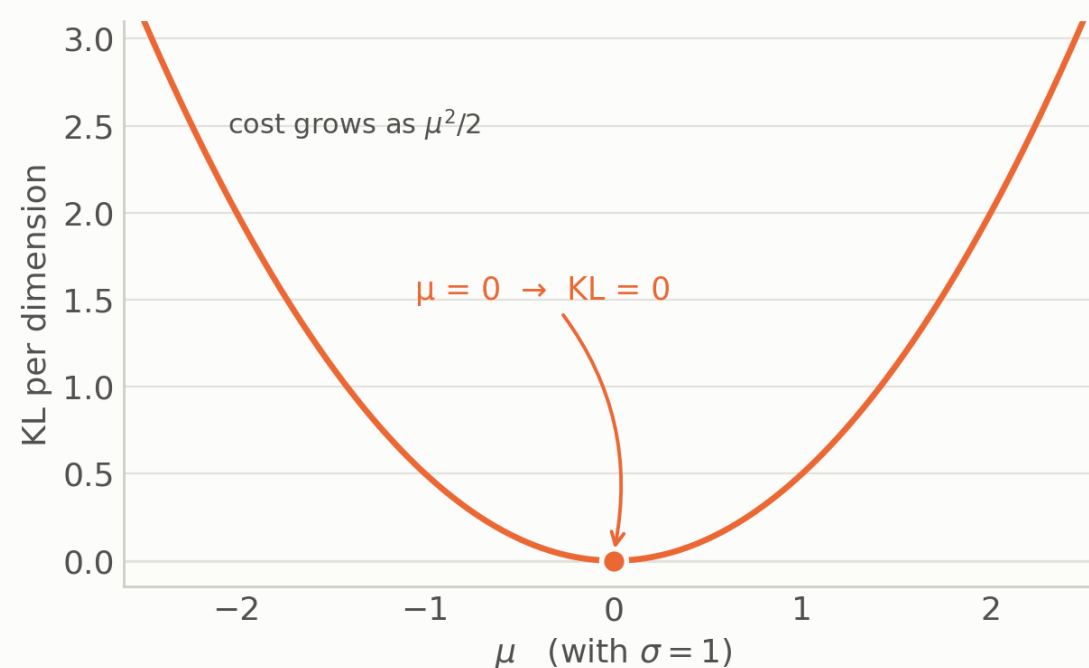
Reading the result

WHAT THE FORMULA PENALISES

The spread is pulled towards 1



The mean is pulled towards 0



The expression is zero exactly when $\mu = 0$ and $\sigma = 1$, and positive everywhere else. The μ^2 term pulls each code towards the centre; the $\sigma^2 - 1 - \log \sigma^2$ term stops it from collapsing to a point or spreading out.

Two conventions that trip people up

BEFORE YOU IMPLEMENT IT

1. σ or σ^2 ?

Some slides write σ where they mean the variance. The formula needs the variance in both slots: σ^2 and $\log \sigma^2$. If your σ is a standard deviation, then $\log \sigma^2 = 2 \log \sigma$.

2. Which side carries the minus?

The KL is never negative. A leading $-\frac{1}{2} \sum (\dots)$ is correct only in the form $-\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$. Written as $-\frac{1}{2} \sum (\sigma^2 + \mu^2 - 1 - \log \sigma^2)$ it is $-D_{\text{KL}}$, not the KL itself.

```
# logvar = log  $\sigma^2$ , straight from the encoder
kl = -0.5 * torch.sum(
    1 + logvar - mu.pow(2) - logvar.exp())
```

Two checks worth running

- KL is 0 when $\mu = 0$, $\log \sigma^2 = 0$.
- KL never prints negative.

The encoder predicts $\log \sigma^2$ rather than σ^2 — it keeps the variance positive automatically and makes the log in the formula free.

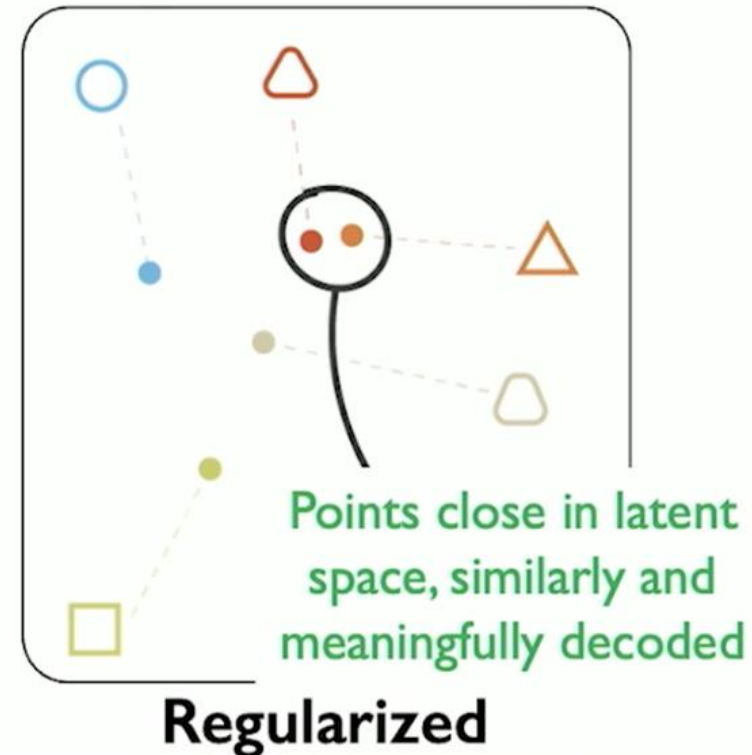
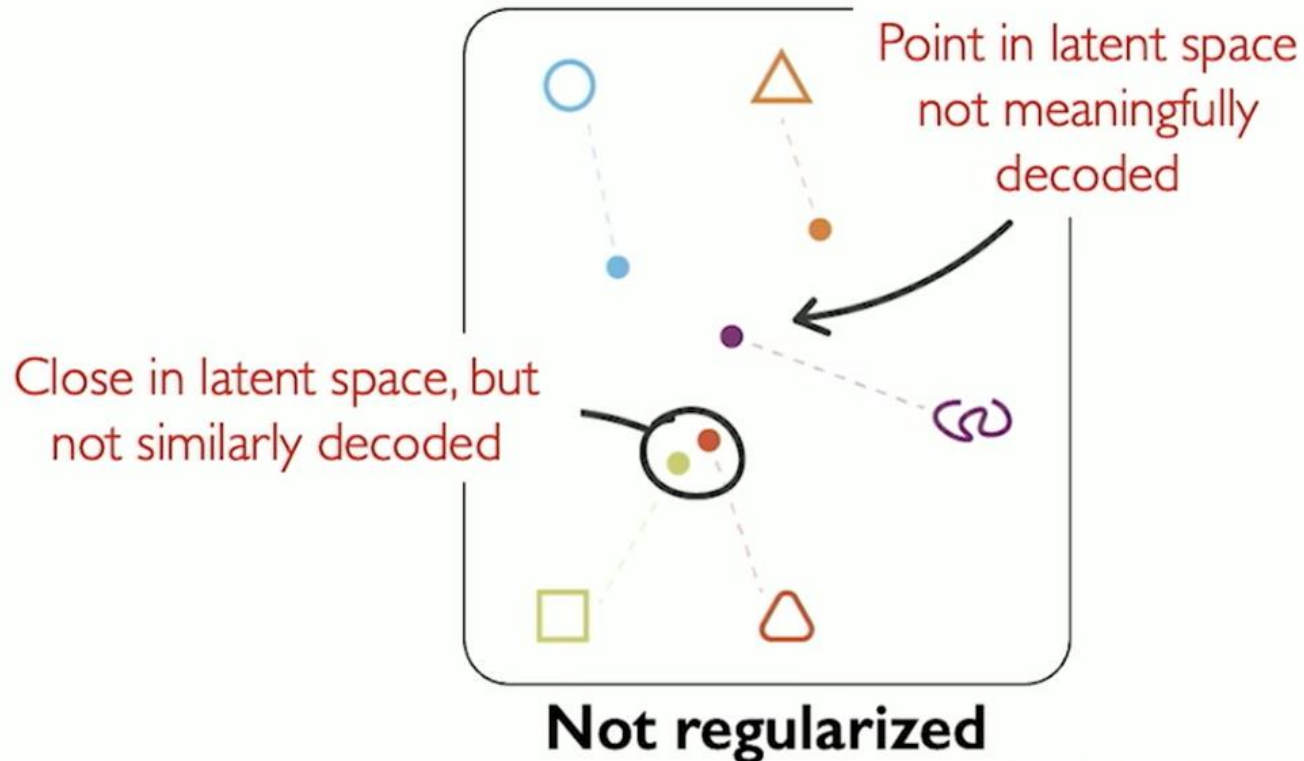
Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

Intuition on regularization and the Normal prior

What properties do we want to achieve from regularization? 🤔

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding



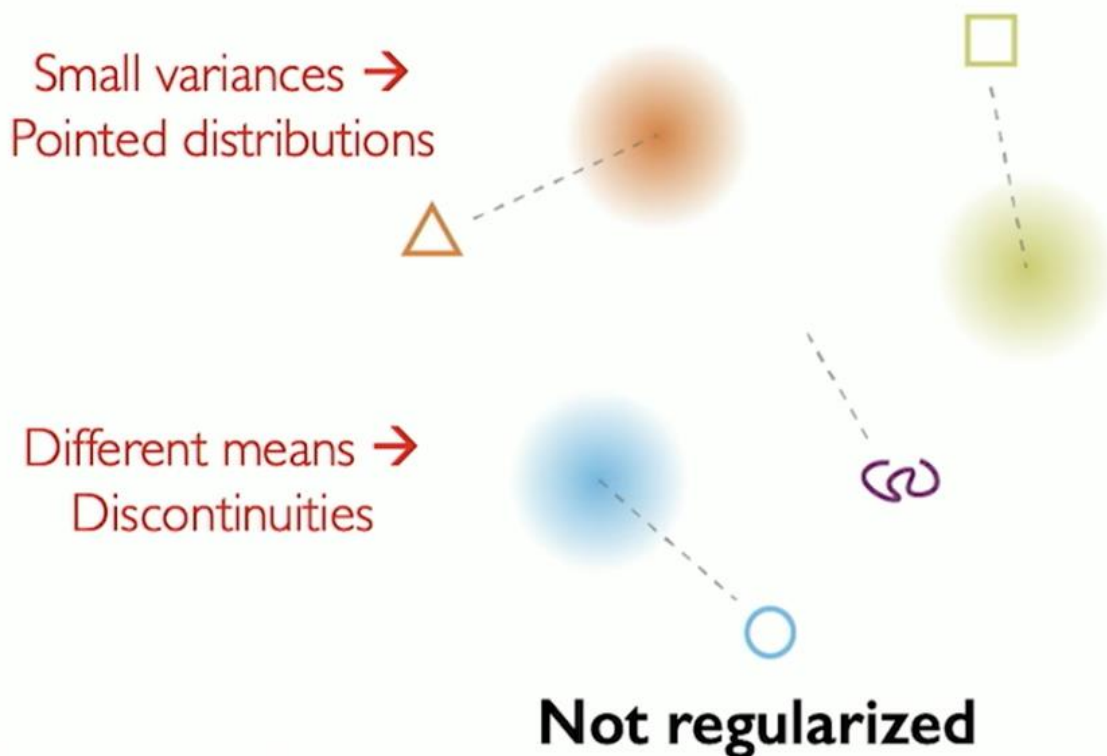
Intuition on regularization and the Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding

Intuition on regularization and the Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding

Encoding as a distribution does not
guarantee these properties!



Intuition on regularization and the Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding

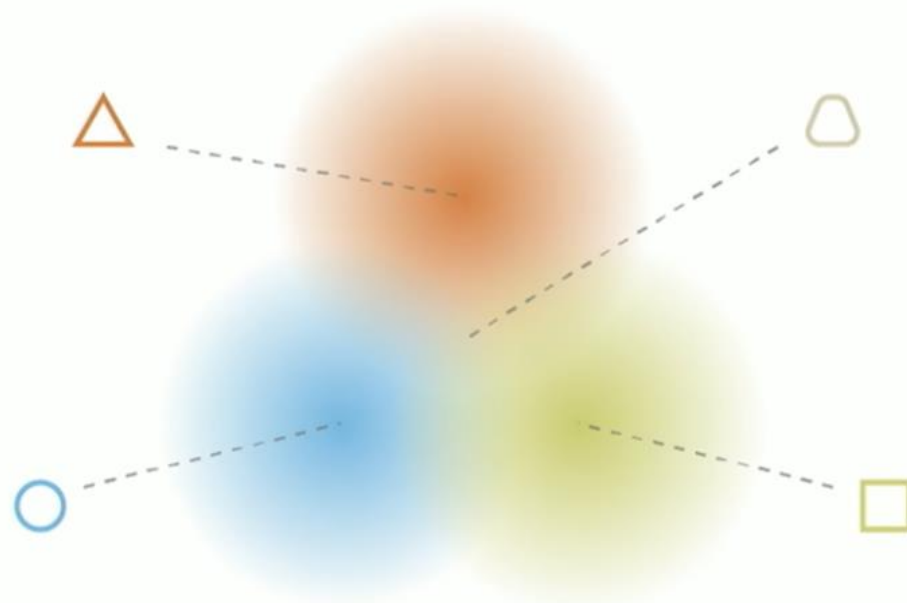
Encoding as a distribution does not
guarantee these properties!

Small variances $\rightarrow$
Pointed distributions

Different means $\rightarrow$
Discontinuities

Not regularized

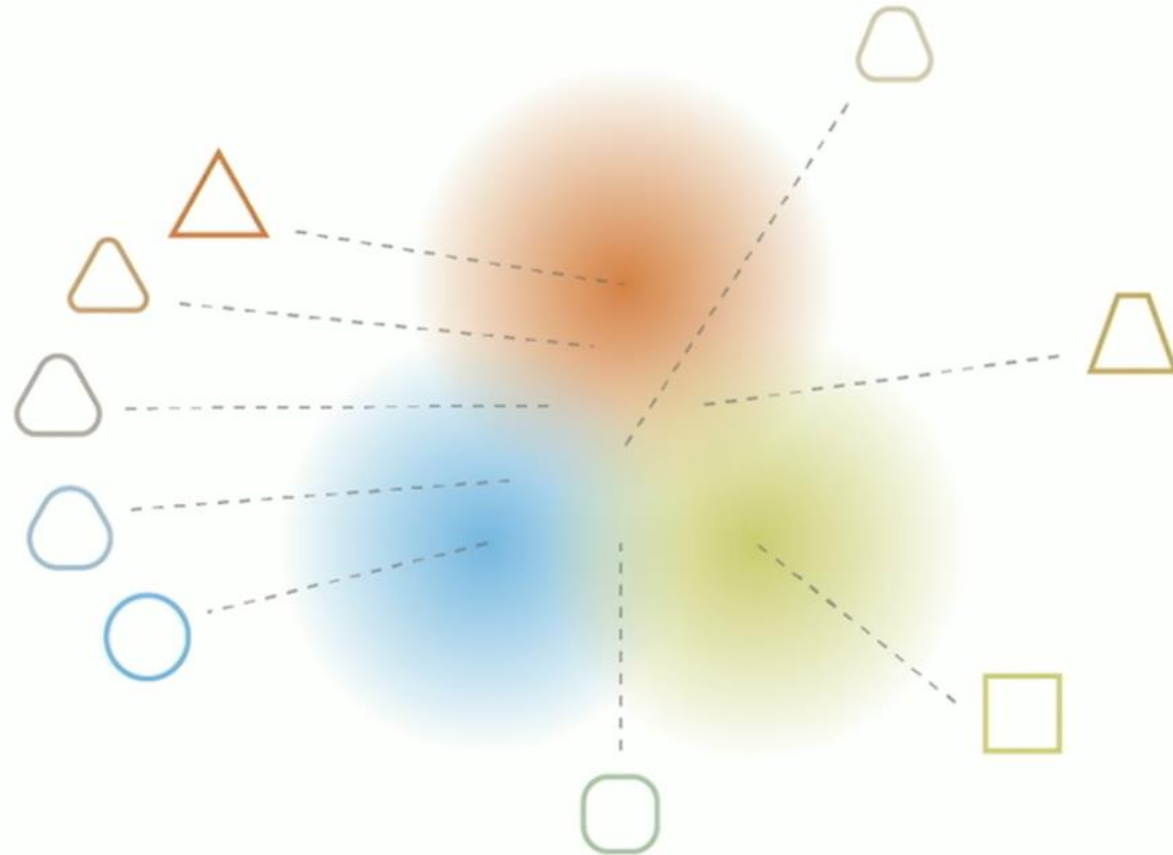
Normal prior $\rightarrow$
continuity + completeness



Regularized

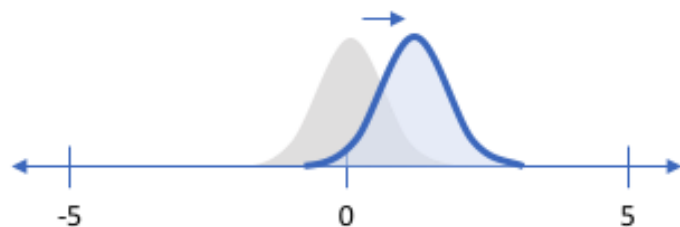
Intuition on regularization and the Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ “meaningful” content after decoding



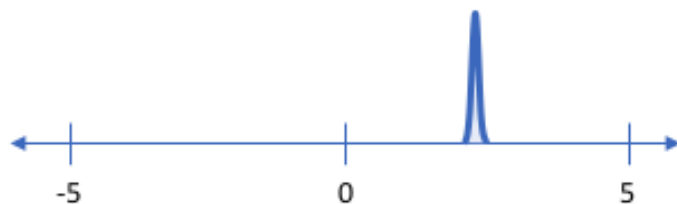
Regularization with Normal prior helps enforce **continuity & completeness** in the latent space.

Penalizing reconstruction loss encourages the distribution to describe the input



Our distribution deviates from the prior to describe some characteristic of the data

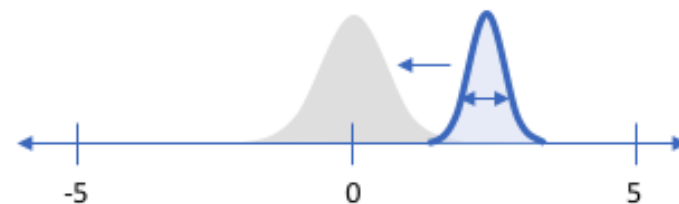
Without regularization, our network can “cheat” by learning narrow distributions



With a small enough variance, this distribution is effectively only representing a single value

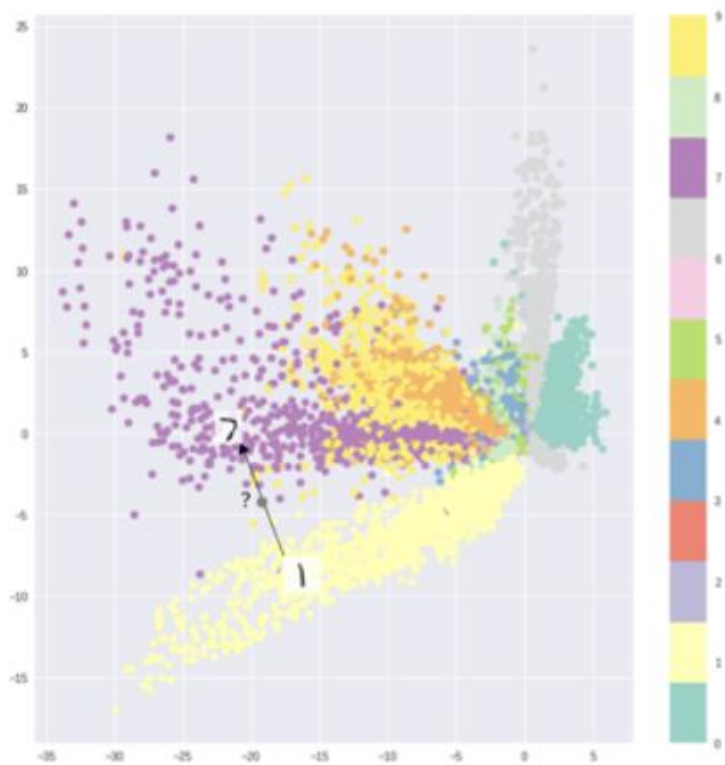
Penalizing KL divergence acts as a regularizing force

Attract distribution to have zero mean

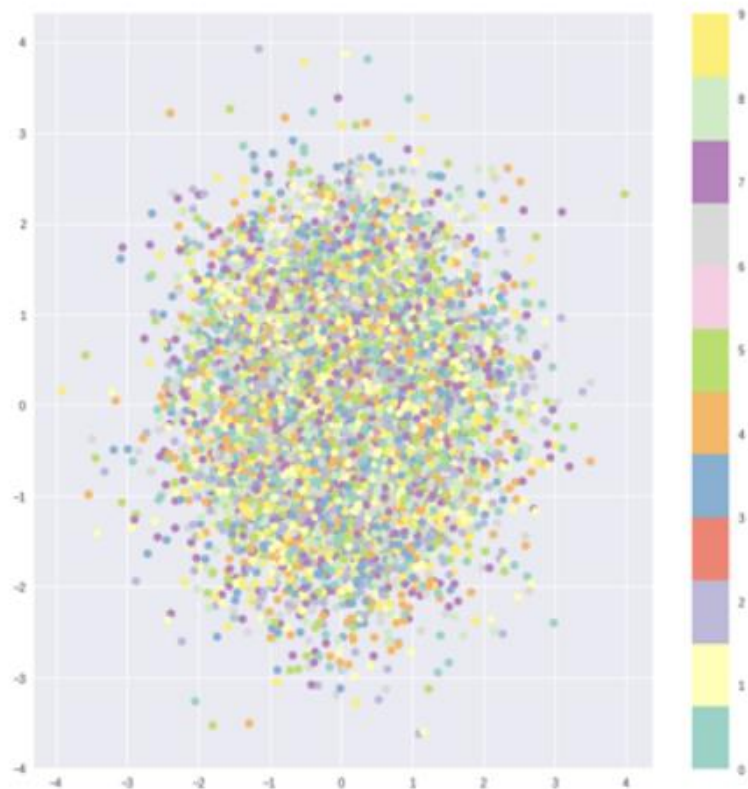


Ensure sufficient variance to yield a smooth latent space

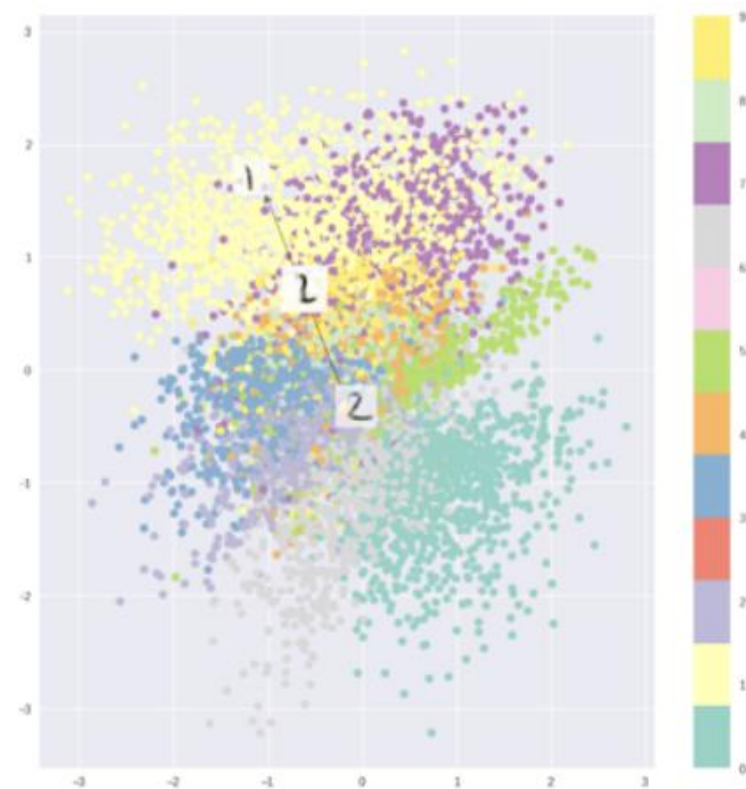
Only reconstruction loss



Only KL divergence

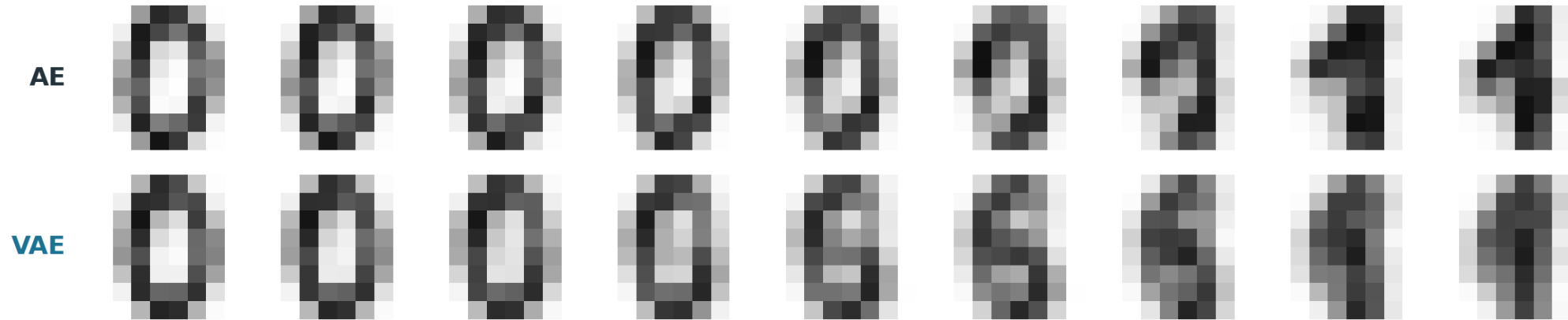


Combination



And a straight line between two images stays meaningful

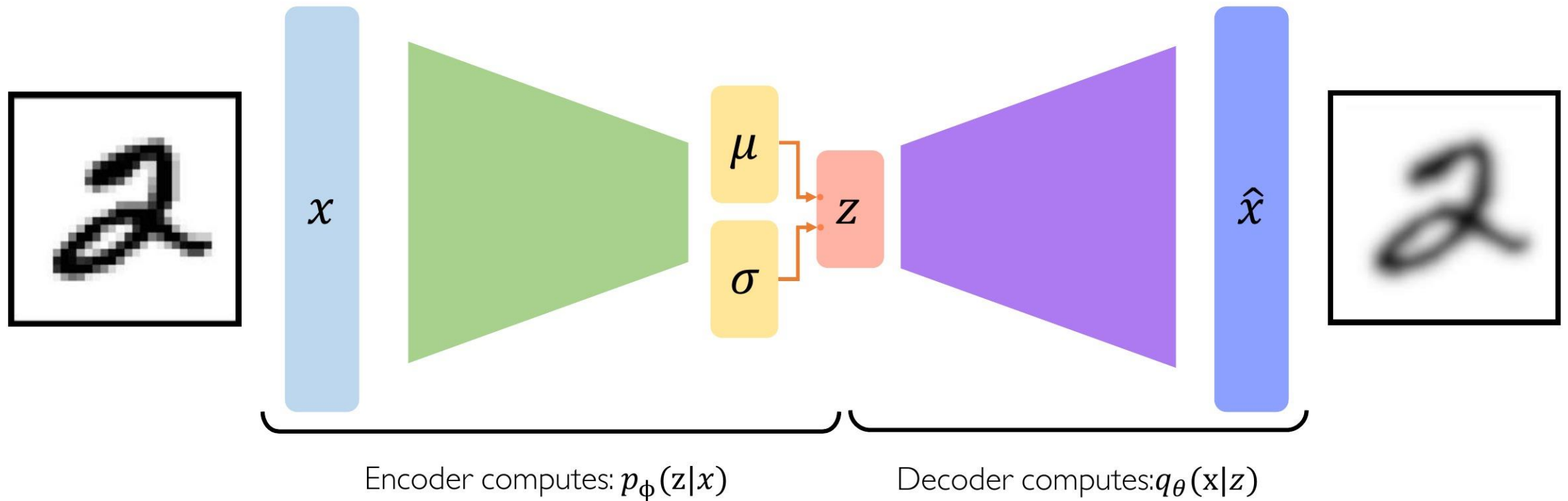
Walk in a straight line from the code of a “0” to the code of a “1”



- AE: holds a “0”, then jumps abruptly. The path crosses a region with no training data in it.
- VAE: a gradual, continuous transition — every intermediate frame is a plausible digit.
- This is the interpolation test from Lecture 1, and it is the cheapest diagnostic you have: **if interpolation looks broken, your latent space is broken.**

You will run exactly this experiment in the assignment, on both of your own trained models.

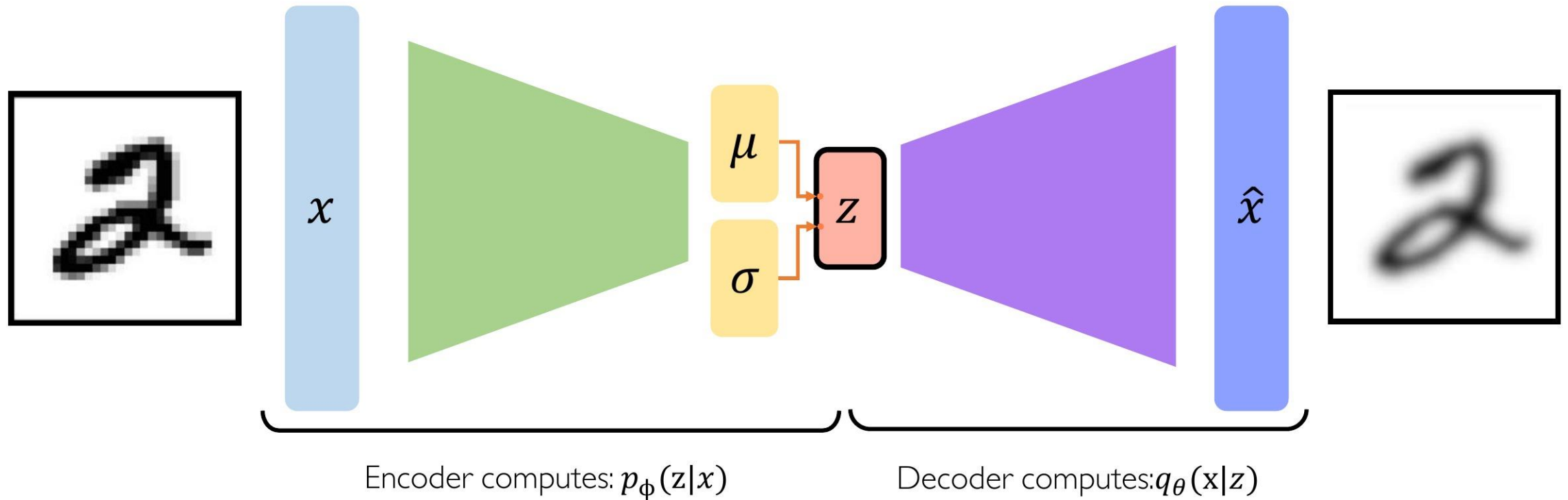
VAEs computation graph



$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

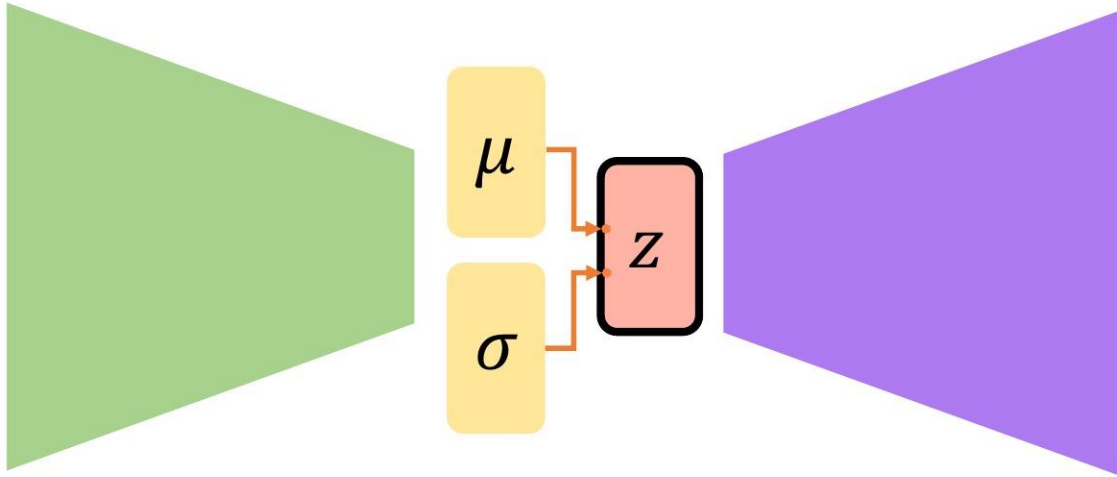
VAEs computation graph

Problem: We cannot backpropagate gradients through sampling layers!



$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

Reparametrizing the sampling layer



Key Idea:

~~$z \sim \mathcal{N}(\mu, \sigma^2)$~~

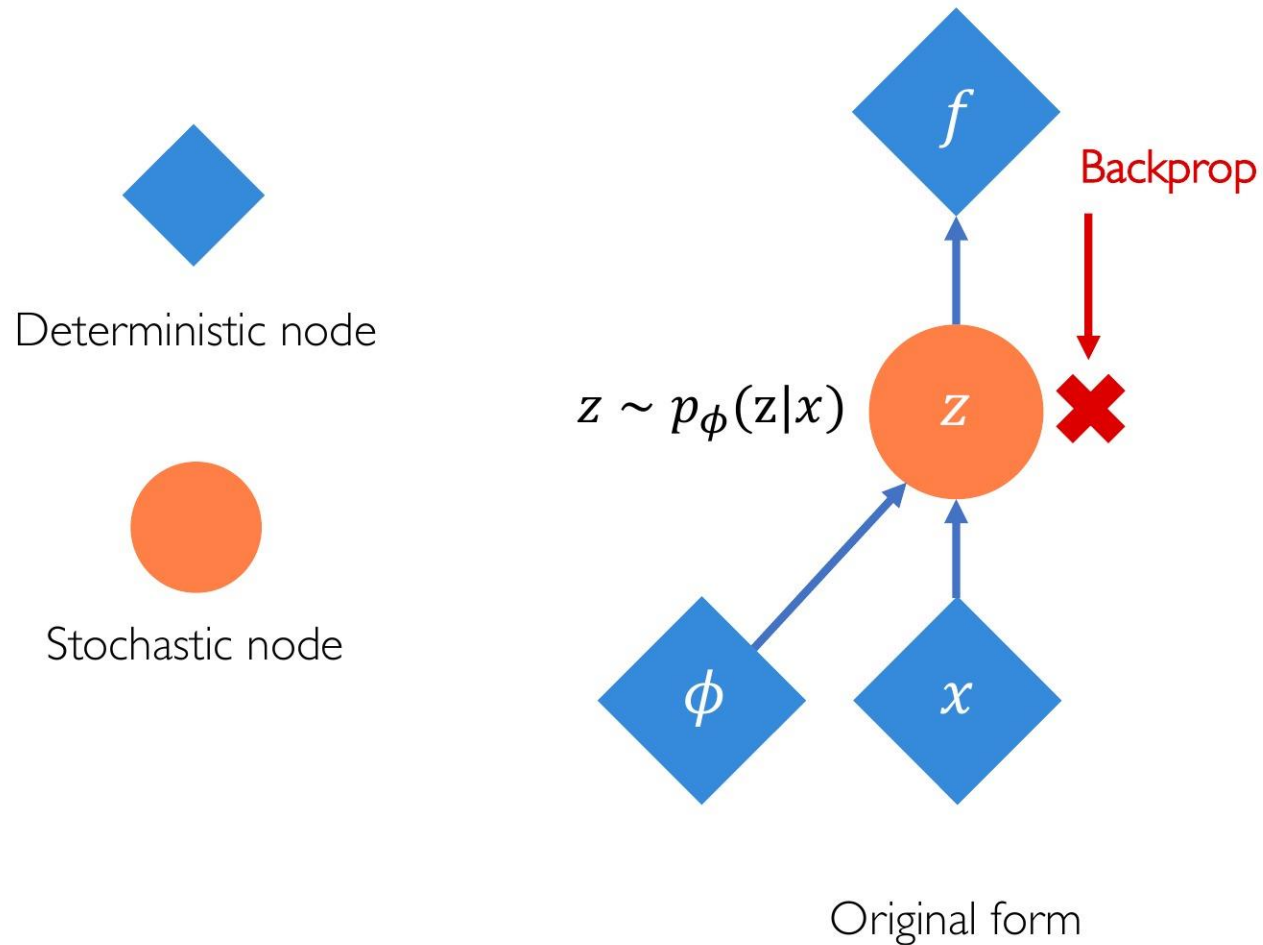
Consider the sampled latent vector as a sum of

- a fixed μ vector,
- and fixed σ vector, scaled by random constants drawn from the prior distribution

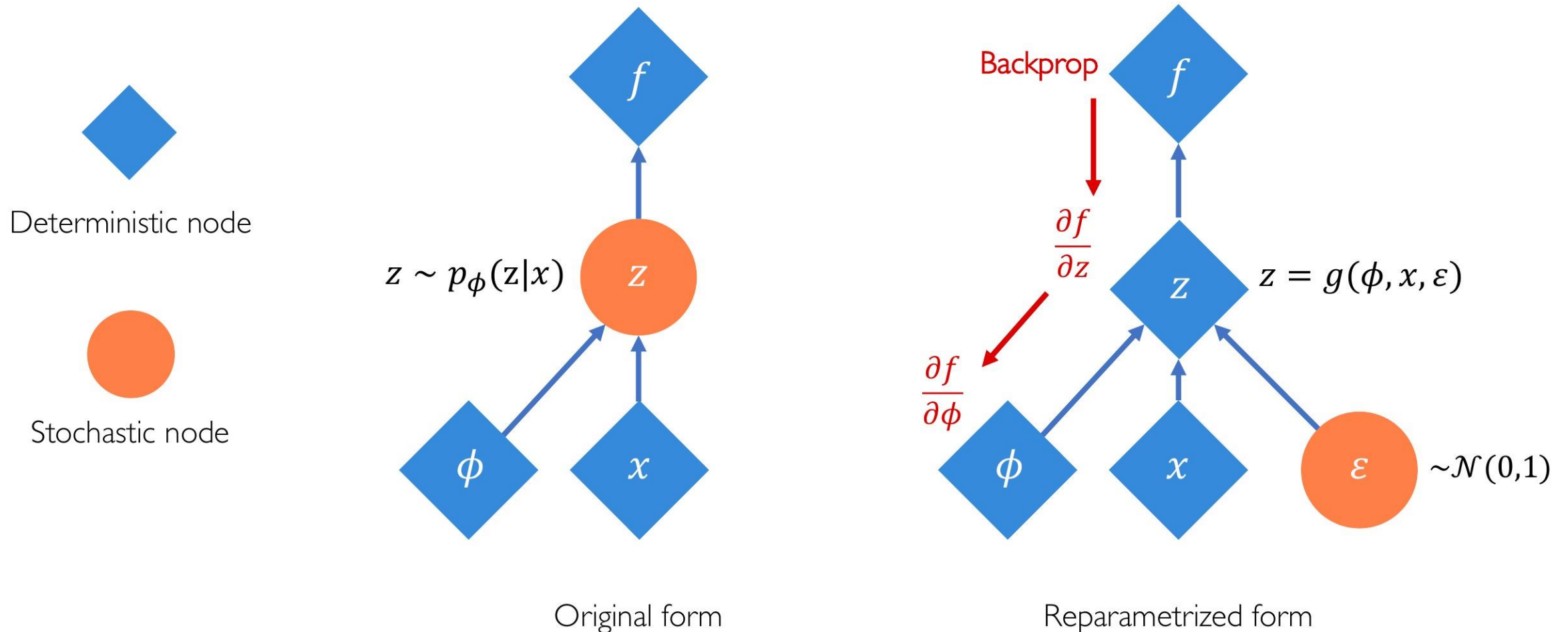
$$\Rightarrow z = \mu + \sigma \odot \varepsilon$$

where $\varepsilon \sim \mathcal{N}(0,1)$

Reparametrizing the sampling layer



Reparametrizing the sampling layer



◆ Step 1: The reparameterization equation

We rewrite the sampling as:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1)$$

Here:

- μ = mean (shift)
- σ = standard deviation (scale)
- ϵ = random noise drawn from a **standard normal distribution**

So instead of directly sampling z , we first sample ϵ and then **shift + scale** it.

◆ Step 2: Why is this called "shift + scale"?

Take a standard normal variable $\epsilon \sim \mathcal{N}(0, 1)$.

- Multiply it by σ : this **scales the spread** of the distribution.
 - If $\sigma = 2$, variance is bigger.
 - If $\sigma = 0.5$, variance is smaller.
- Add μ : this **shifts the center** of the distribution.
 - If $\mu = 5$, the bell curve moves to be centered at 5.
 - If $\mu = -3$, the bell curve moves left.

So mathematically:

$$z = \mu + \sigma \epsilon \quad \Rightarrow \quad z \sim \mathcal{N}(\mu, \sigma^2)$$

◆ Step 3: Differentiability

Now let's check gradients.

$$z = \mu + \sigma \epsilon$$

- Derivative wrt μ :

$$\frac{\partial z}{\partial \mu} = \frac{\partial}{\partial \mu}(\mu + \sigma \epsilon) = 1$$

- Derivative wrt σ :

$$\frac{\partial z}{\partial \sigma} = \frac{\partial}{\partial \sigma}(\mu + \sigma \epsilon) = \epsilon$$

◆ Step 4: What does this mean practically?

- Without reparameterization $\rightarrow z$ is sampled randomly, so no derivatives exist.
- With reparameterization $\rightarrow z$ is a deterministic function of μ , σ , and random ϵ .
 - Gradient wrt $\mu = 1$ (so updates flow directly).
 - Gradient wrt $\sigma = \epsilon$ (so updates depend on sampled noise).

VAEs: Latent perturbation

Slowly increase or decrease a **single latent variable**
Keep all other variables fixed



Head pose

Different dimensions of z encodes **different interpretable latent features**

http://dpkingma.com/sgvb_mnist_demo/demo.html

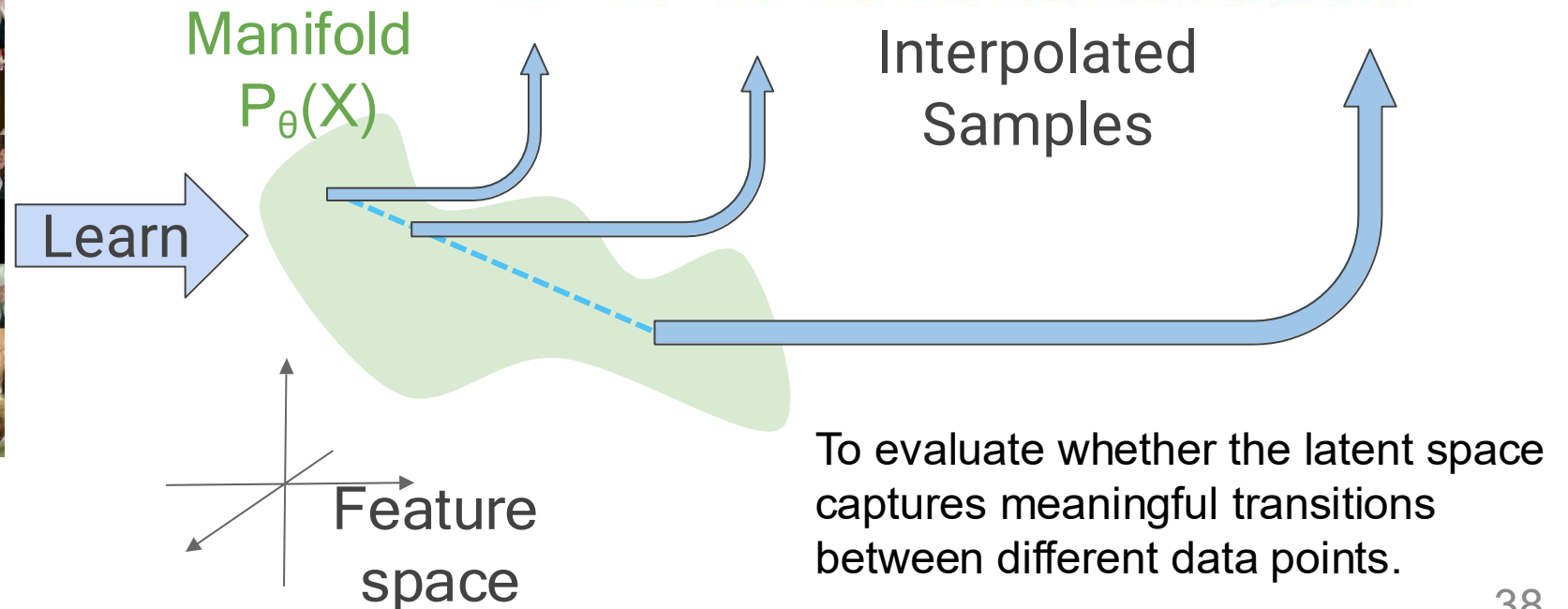
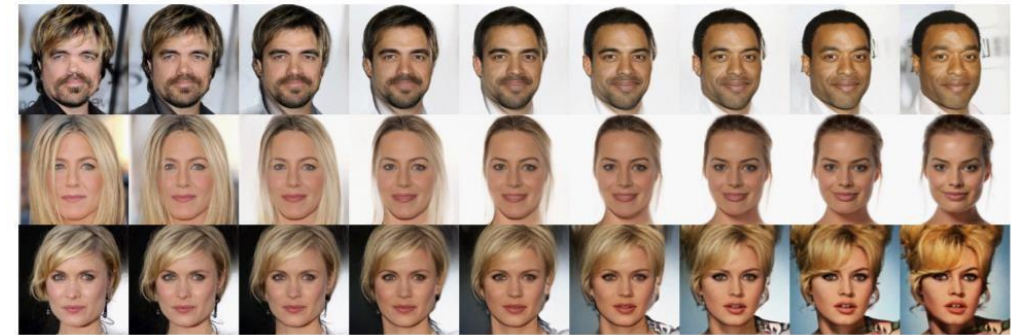
Interpolation

Interpolation is the process of smoothly transitioning between two points in the latent space to generate intermediate outputs.

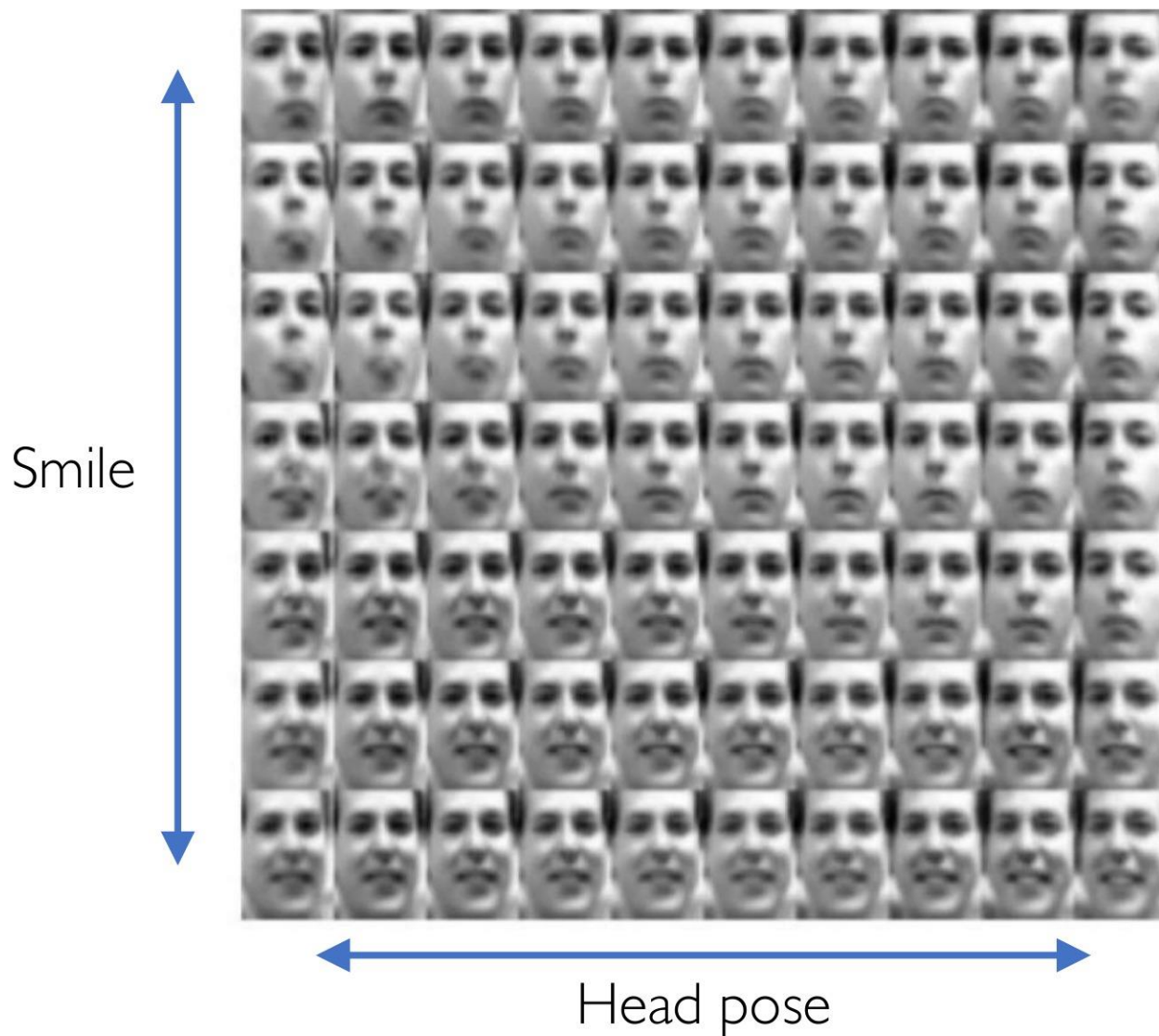
Traversing the learned manifold through interpolation.



Training Dataset



VAEs: Latent perturbation



Ideally, we want latent variables that are uncorrelated with each other

Enforce diagonal prior on the latent variables to encourage independence

Disentanglement

Why latent variable models? Debiasing

Capable of uncovering **underlying latent variables** in a dataset



Homogeneous skin color, pose

VS



Diverse skin color, pose, illumination

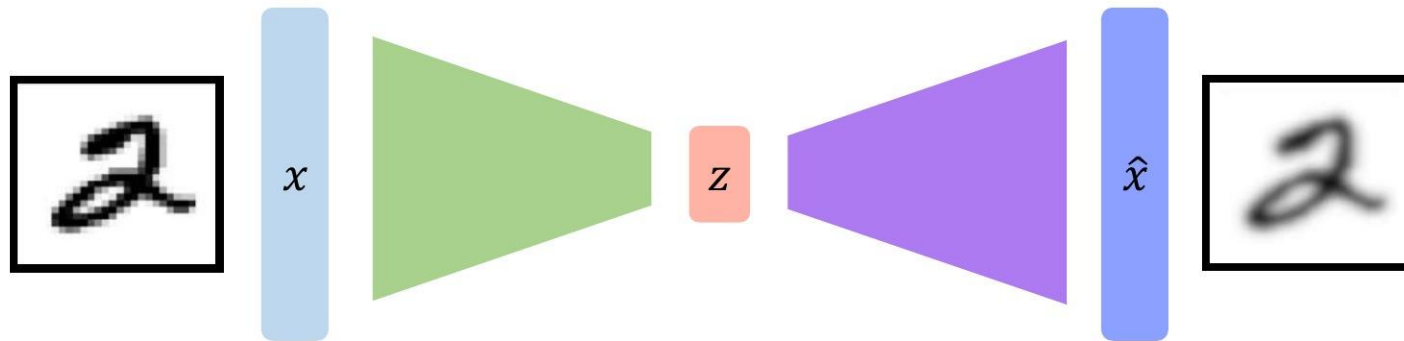
How can we use latent distributions to create fair and representative datasets?



Software Lab!

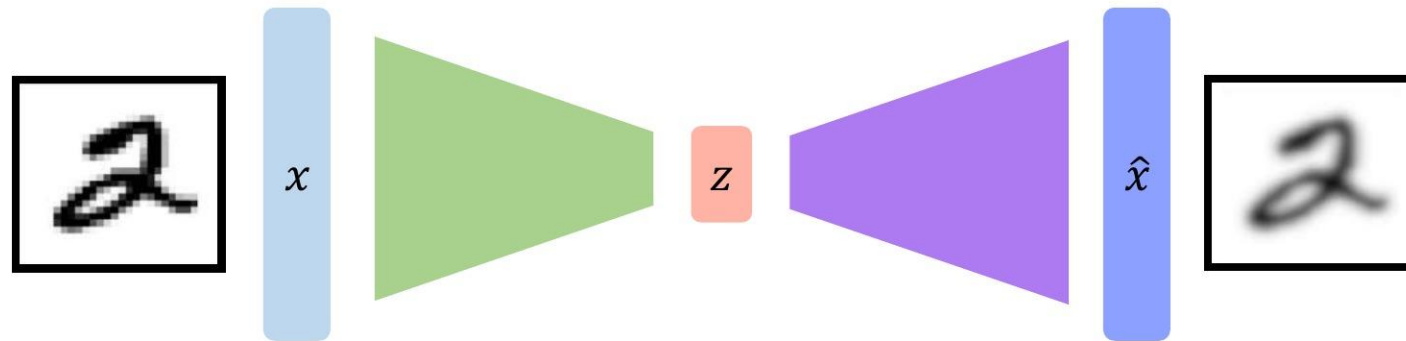
VAE summary

- I. Compress representation of world to something we can use to learn



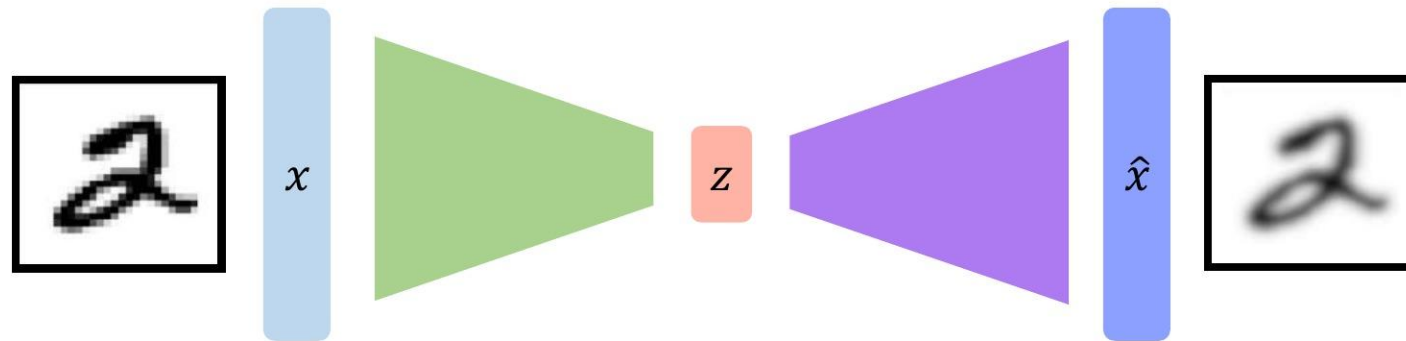
VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)



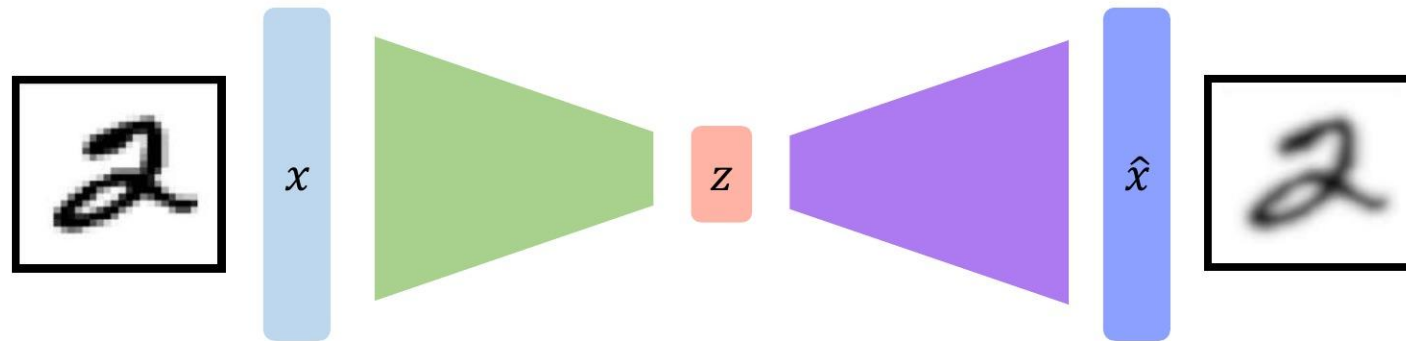
VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end



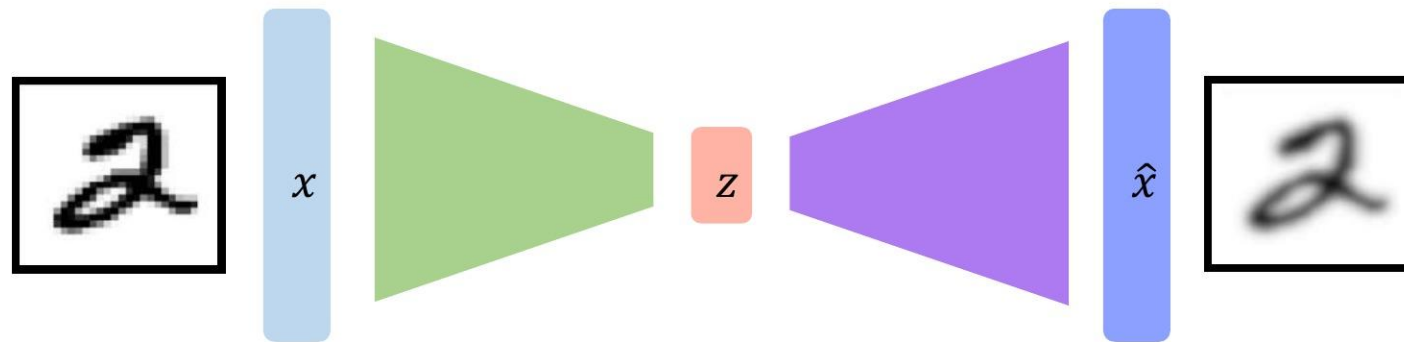
VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end
4. Interpret hidden latent variables using perturbation



VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end
4. Interpret hidden latent variables using perturbation
5. Generating new examples



Resources

<https://towardsdatascience.com/variational-autoencoders-vaes-explained-from-theory-to-elbo-and-the-reparameterization-trick/>

<https://www.jeremyjordan.me/variational-autoencoders/>